Orange Pi 5B + лидар YDLIDAR X3 Pro

Инструкция по подготовке одноплатного компьютера к работе с лазерным дальномером. Установка операционной системы, перенос во встроенную память, обновление, точка доступа Wi-Fi, удалённый рабочий стол, подключение лидара, ROS 2, визуализация в RViz2, построение карты помещения (SLAM) и клонирование готовой системы на другие платы.

1. Что понадобится

Оборудование из комплекта:

- одноплатный компьютер **Orange Pi 5B** (процессор Rockchip RK3588S, встроенная память eMMC);
- лазерный дальномер (лидар) **YDLIDAR X3 Pro** с платой-адаптером последовательного порта и кабелем USB Type-C.

Дополнительно:

- блок питания USB Type-C **5 В / 4 А** — желательно простой, без «быстрой зарядки»: некоторые PD-зарядки плата не запускает;
- карта **microSD** объёмом от 16 ГБ (желательно класса A1/A2) и картридер для компьютера;
- **Ethernet-кабель** для подключения платы к роутеру (на первом запуске Wi-Fi ещё не настроен);
- компьютер с **Windows** в той же локальной сети;
- радиатор или кулер для процессора — без охлаждения RK3588 под нагрузкой снижает частоту.

Программы на компьютере с Windows:

Программа	Для чего	Где взять
7-Zip	Распаковка образа системы (.7z)	7-zip.org
balenaEtcher	Запись образа на карту microSD	etcher.balena.io
Bitvise SSH Client	Подключение к плате по SSH и обмен файлами (SFTP)	bitvise.com
Advanced IP Scanner (по желанию)	Поиск IP-адреса платы в сети	advanced-ip-scanner.com

> **Монитор не обязателен.** Всю настройку можно выполнить по сети через SSH. Монитор с HDMI удобен только для первой проверки, что система загрузилась.

2. Выбор и скачивание образа системы

На встроенной памяти eMMC платы с завода установлен **Android** (при включении без карты загружается оболочка с сервисами Google). Заводской загрузчик Android мешает загрузке сторонних образов (например, Armbian) с карты: плата просто не стартует, горит только красный светодиод. Поэтому используем **официальный образ Orange Pi** — он загружается с карты без дополнительных действий.

1. Откройте сайт **orangepi.org** → раздел **Orange Pi 5B** → **Official Images**.
2. Выберите плитку **Ubuntu Image** и нажмите **Downloads** (рис. 1).
3. В списке файлов выберите образ: версия Ubuntu **jammy** (22.04), вариант **desktop_xfce**, ядро **linux6.1**. Имя файла: `Orangepi5b_1.1.0_ubuntu_jammy_desktop_xfce_linux6.1.99.7z`.

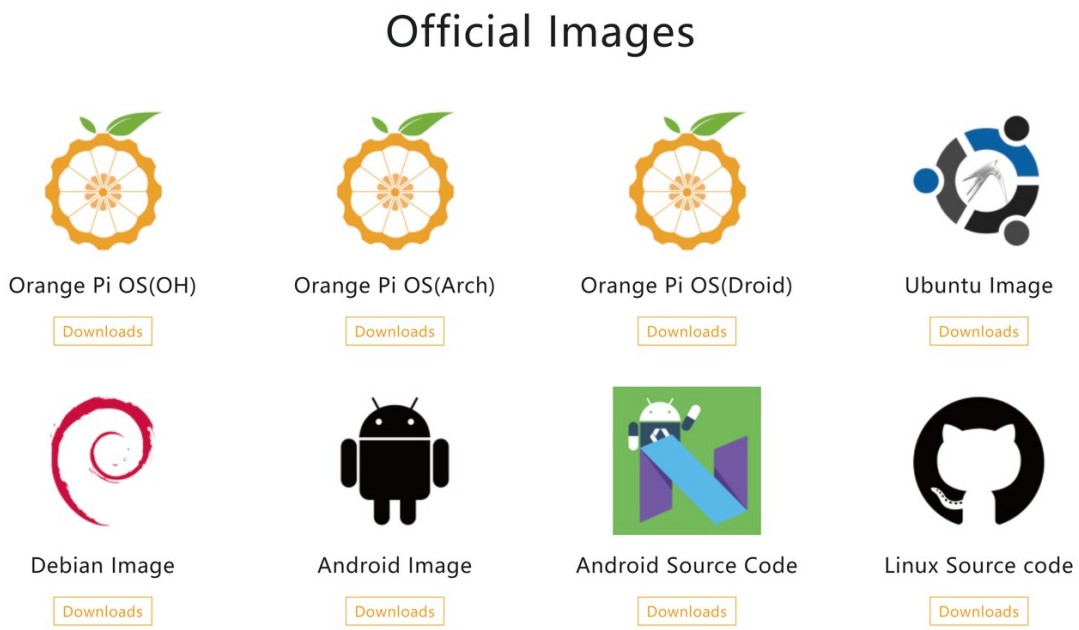


Рис. 1. Страница официальных образов Orange Pi 5B. Нужна плитка «Ubuntu Image»

Почему именно этот вариант:

- **jammy (Ubuntu 22.04)** — самый проверенный вариант для 5B, под него есть ROS 2 Humble с готовыми пакетами, драйвер лидара YDLIDAR собирается без проблем;
- **desktop_xfce** — с лёгким рабочим столом Xfce: он понадобится для визуализации данных лидара (RViz) и легко подключается по RDP из Windows;
- **linux6.1** — более новое ядро от Rockchip с лучшей поддержкой оборудования (если есть выбор с linux5.10).

3. Запись образа на карту microSD

1. Распакуйте скачанный архив `.7z` программой **7-Zip** (правая кнопка → 7-Zip → «Распаковать здесь»). Внутри будет файл `.img` размером около 7 ГБ — именно его записываем. balenaEtcher не умеет открывать архивы `.7z`.
2. Вставьте карту microSD в картридер компьютера.
3. Запустите **balenaEtcher** → **Flash from file** → выберите файл `.img`.
4. **Select target** → выберите карту microSD. Внимательно проверьте объём, чтобы не выбрать другой диск!
5. Нажмите **Flash!** и дождитесь сообщения **Flash Complete** после проверки (Validating). Это 5–10 минут.

> **Важно.** Если после записи Windows предложит «отформатировать диск» — нажмите **Отмена**. Windows не понимает разделы Linux, это нормально.

4. Первый запуск с карты microSD

1. Вставьте карту в слот microSD на нижней стороне платы — контактами к плате, до упора.
2. Подключите Ethernet-кабель от платы к роутеру (и при желании монитор по HDMI).
3. Подключите питание в разъём питания USB Type-C. На плате два разъёма Type-C — второй предназначен для USB/видео.
4. Подождите 1–2 минуты.

Как понять, что система загрузилась (по светодиодам):

Светодиоды	Что это значит	Что делать
Ничего не горит	Нет питания	Проверить блок питания, кабель и разъём
Горит только красный	Питание есть, но система не стартует	Перезаписать карту, попробовать другую карту или другой образ (см. раздел 2)
Красный + мигает зелёный	Система работает	Найти IP-адрес и подключиться по SSH

Как узнать IP-адрес платы:

- в веб-интерфейсе роутера, в списке подключённых клиентов (устройство `orangepi5b`);
- программой Advanced IP Scanner;
- если подключён монитор — командой `hostname -I` в терминале на плате.

В нашем случае адрес платы — **192.168.30.33**. Дальше в инструкции используется он, подставьте свой.

> **Учётные записи официального образа:** пользователь `orangepi`, пароль `orangepi`; администратор `root`, пароль `orangepi`.

5. Подключение по SSH через Bitvise

SSH-сервер в официальном образе включён по умолчанию, поэтому дальше всё делается удалённо с компьютера Windows. Используем **Bitvise SSH Client** — у него есть терминал и окно SFTP для копирования файлов мышкой.

1. Запустите Bitvise SSH Client, вкладка **Login** (рис. 2).
2. Поле **Host** — IP-адрес платы (`192.168.30.33`), **Port** — `22`.
3. **Username** — `root` (или `orangepi`), **Initial method** — `password`, **Password** — `orangepi`. Если в поле пароля уже что-то сохранено от другого сервера — сотрите.
4. Нажмите **Log in**. При первом подключении появится окно с ключом сервера (Host key) — нажмите **Accept and Save**.
5. Откроются два окна: **терминал** (для команд) и **SFTP** (для файлов).
6. Сохраните настройки кнопкой **Save profile as** (например, `orangepi5b.tlp`), чтобы в следующий раз входить одной кнопкой.

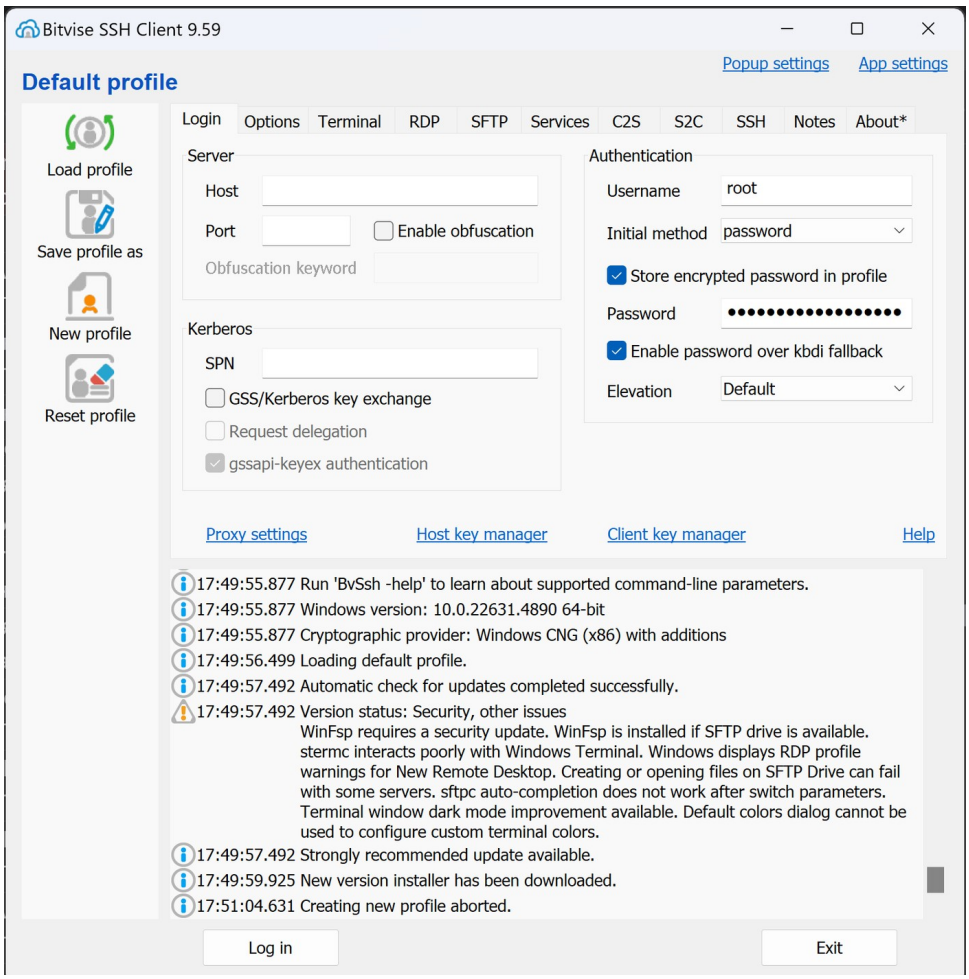


Рис. 2. Окно Bitvise SSH Client, вкладка Login

> **Смена ключа сервера.** После переустановки системы (раздел 6) у платы будет новый ключ, и Bitvise покажет предупреждение. Это ожидаемо — просто примите новый ключ.

6. Перенос системы на встроенную память eMMC

Пока система работает **с карты microSD**, а на eMMC по-прежнему стоит Android. Если вынуть карту, плата загрузит Android. Переносим систему на eMMC: она быстрее и надёжнее карты и не выпадет из слота от вибрации на роботе.

> **Внимание!** Всё содержимое eMMC (заводской Android) будет удалено.

6.1. Копирование образа на плату

Перетащите файл `.img` из папки Windows в окно **SFTP** Bitvise, в домашнюю папку `/home/orangepi` (рис. 3). Копирование 7 ГБ по сети занимает несколько минут.

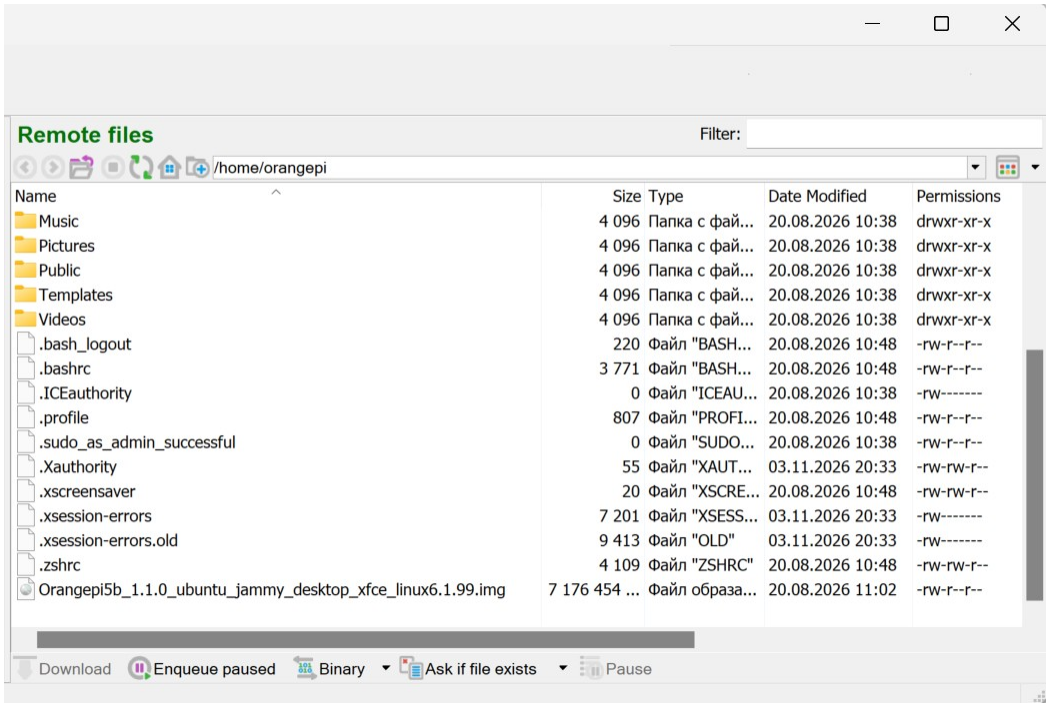


Рис. 3. Образ системы скопирован на плату в папку /home/orangepi

6.2. Определение устройства eMMC

В терминале выполните:

```
lsblk
```

```
root@orangepi5b:~# lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
mmcblk0      179:0    0 29.1G  0 disk
├─mmcblk0p1  179:1    0    4M  0 part
├─mmcblk0p2  179:2    0    6M  0 part
├─mmcblk0p3  179:3    0    4M  0 part
├─mmcblk0p4  179:4    0    4M  0 part
├─mmcblk0p5  179:5    0    4M  0 part
├─mmcblk0p6  179:6    0    1M  0 part
├─mmcblk0p7  179:7    0  100M  0 part
├─mmcblk0p8  179:8    0  108M  0 part
├─mmcblk0p9  179:9    0  372M  0 part
├─mmcblk0p10 179:10   0  384M  0 part
├─mmcblk0p11 179:11   0   16M  0 part
├─mmcblk0p12 179:12   0    1M  0 part
├─mmcblk0p13 179:13   0    3G  0 part
└─mmcblk0p14 179:14   0 25.1G  0 part
mmcblk0boot0 179:32   0    4M  1 disk
mmcblk0boot1 179:64   0    4M  1 disk
mmcblk1      179:96   0 58.3G  0 disk
└─mmcblk1p1  179:97   0 57.6G  0 part /var/log.hdd
                                      /
zram0        254:0    0  1.9G  0 disk [SWAP]
zram1        254:1    0  200M  0 disk /var/log
root@orangepi5b:~#
```

Рис. 4. Вывод lsblk до переноса: mmcblk0 — eMMC с разделами Android, mmcblk1 — карта microSD

Как отличить устройства (рис. 4):

- **mmcblk0** (29,1 ГБ) — это **eMMC**: на нём 14 разделов Android, рядом есть служебные `mmcblk0boot0` и `mmcblk0boot1`. **Сюда пишем.**

- **mmcblk1** (58,3 ГБ) — это **карта microSD**: у её раздела в столбце MOUNTPOINTS стоит `/` — с неё сейчас работает система. **Её не трогаем.**

> **Осторожно!** Если перепутать устройства, будет стёрта карта, с которой работает система. Если есть сомнения — сверьтесь с размерами и наличием `boot0`/`boot1`.

6.3. Запись образа на eMMC

Под пользователем `root` выполните (одна команда, путь к образу полный):

```
dd if=/home/orangepi/Orangepi5b_1.1.0_ubuntu_jammy_desktop_xfce_linux6.1.99.img \
   of=/dev/mmcblk0 bs=4M status=progress conv=fsync
sync
```

Команда `dd` побайтно копирует образ на eMMC, `status=progress` показывает ход записи, `conv=fsync` и `sync` гарантируют, что данные физически записаны. Запись занимает 2–5 минут — дождитесь, пока снова появится приглашение `root@orangepi5b:~#`.

6.4. Загрузка с eMMC

1. Выключите плату командой `poweroff`.
2. Дождитесь, пока погаснет зелёный светодиод, и отключите питание.
3. **Выньте карту microSD.**
4. Включите плату. Первая загрузка чуть дольше обычного: система автоматически расширяет раздел на весь объём eMMC.
5. Подключитесь в Bitvise заново (логин и пароль снова по умолчанию, примите новый ключ сервера).

Проверьте результат командой `lsblk` (рис. 5): корень системы `/` теперь находится на **mmcblk0p1** (28,8 ГБ), а карты в списке нет.

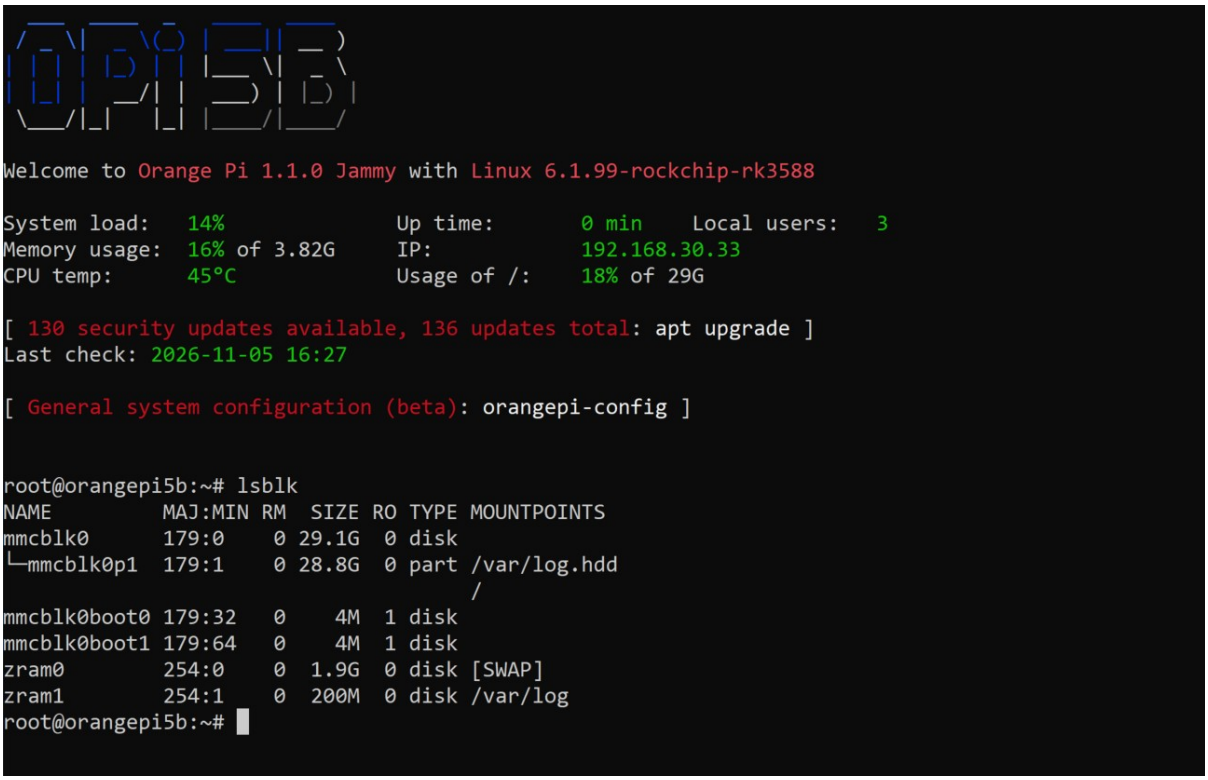


Рис. 5. Система загружена с eMMC: корень «/» на mmcblk0p1, раздел расширен до 28,8 ГБ

Приветствие при входе показывает версию системы (Orange Pi 1.1.0 Jammy, ядро Linux 6.1.99), IP-адрес, температуру процессора и количество доступных обновлений.

Карта microSD теперь свободна, её можно очистить и использовать как хранилище. Не оставляйте в плате карту с загрузочным образом — иначе плата снова загрузится с неё.

7. Часовой пояс и обновление системы

7.1. Часовой пояс

По умолчанию в образе стоит китайский часовой пояс (время на 5 часов впереди московского). Исправляем:

```
timedatectl set-timezone Europe/Moscow
```

7.2. Ошибка репозитория Docker

Первая попытка обновления завершилась ошибкой (рис. 6):

```
apt update && apt upgrade -y
```

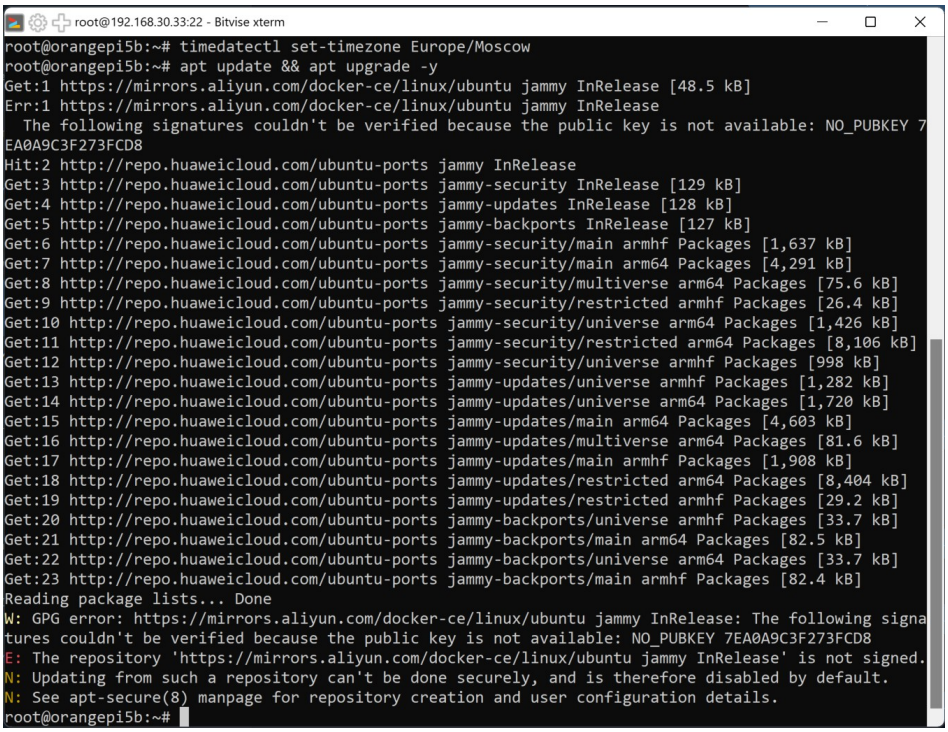


Рис. 6. Ошибка «NO_PUBKEY … is not signed» для репозитория Docker на зеркале aliyun

Причина — в образ добавлен репозиторий **Docker** с китайского зеркала `mirrors.aliyun.com`, для которого нет ключа подписи. Из-за строки с ошибкой `E:` команда `apt update` завершается неудачно, и `apt upgrade` не запускается. Docker для работы с лидаром не нужен — удаляем этот репозиторий:

```
grep -l docker-ce /etc/apt/sources.list.d/* | xargs -r rm -v
```

Команда находит файл с репозиторием Docker и удаляет его — в ответ выводится `removed '/etc/apt/sources.list.d/docker.list'` (рис. 7).

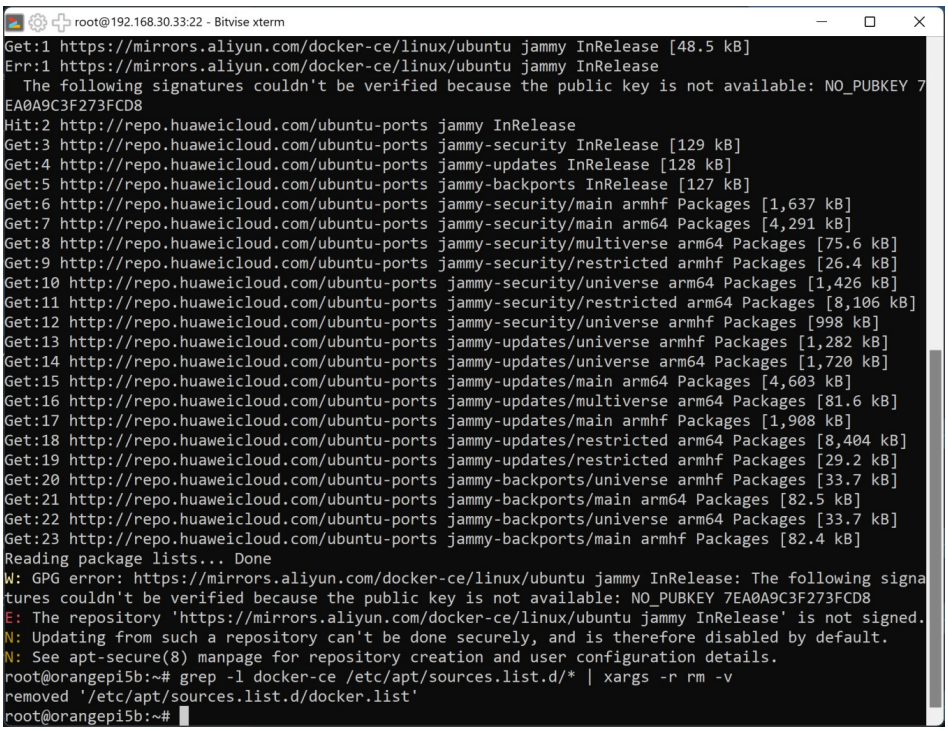


Рис. 7. Файл docker.list удалён

По желанию можно заменить китайское зеркало Huawei на зеркало Яндекса — из России оно обычно быстрее:

```
sed -i 's|http://repo.huaweicloud.com/ubuntu-ports|http://mirror.yandex.ru/ubuntu-ports|g' /etc/apt/sources.list
```

7.3. Обрыв SSH-соединения во время обновления

При повторном запуске `apt update && apt upgrade -y` соединение оборвалось — окно Bitvise показало **[disconnected]** (рис. 8).

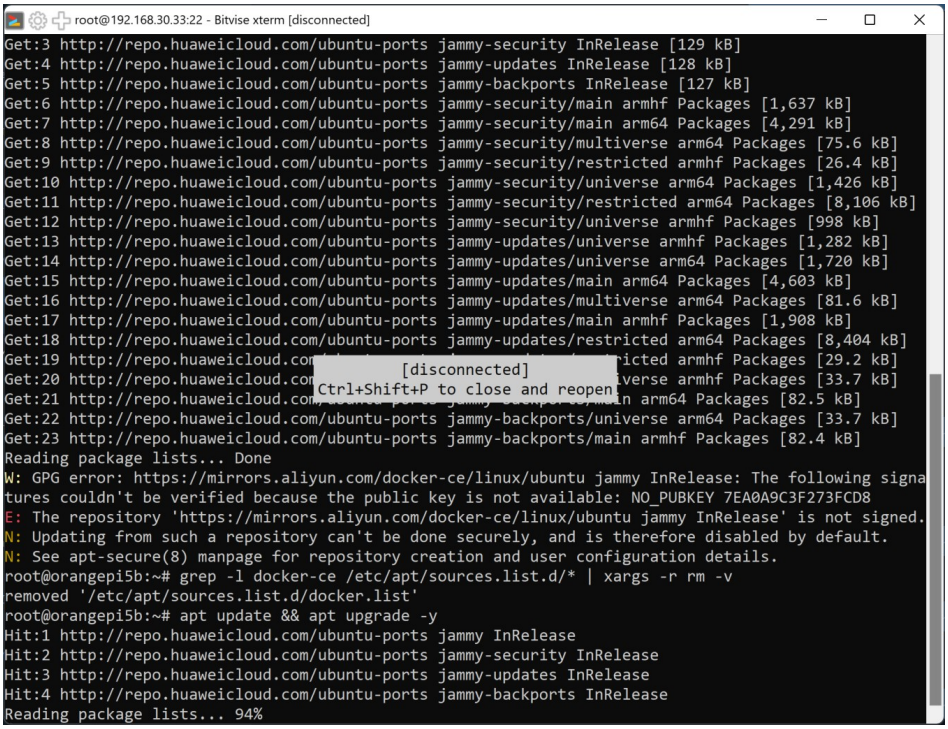


Рис. 8. Соединение SSH оборвалось во время обновления

Чтобы понять причину, переподключитесь и выполните:

```
uptime
```

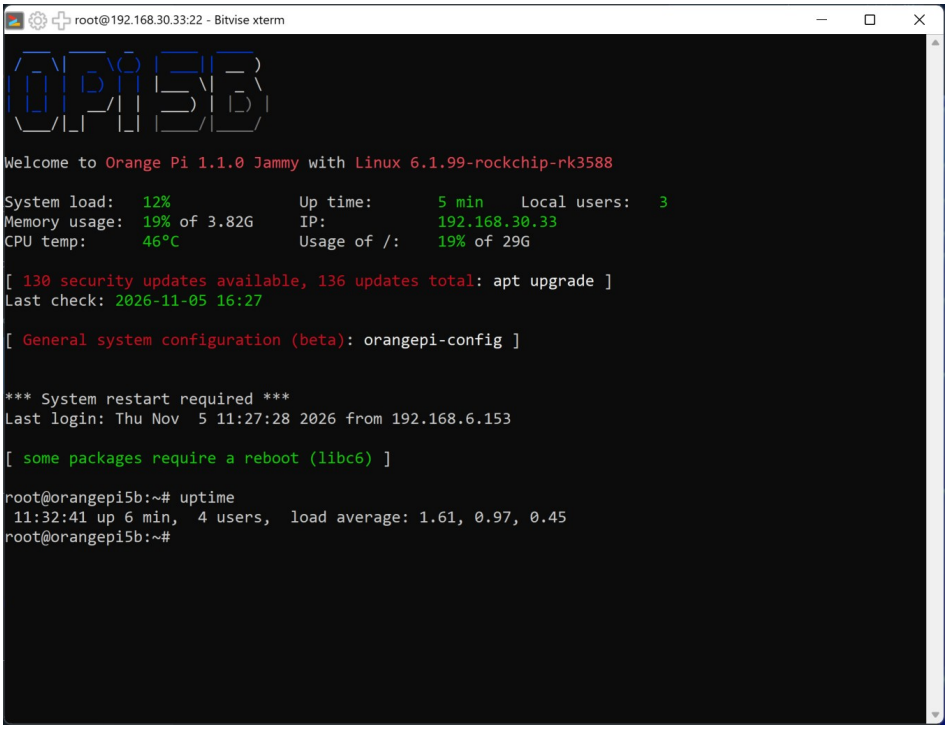


Рис. 9. uptime: плата работает 6 минут — перезагрузки не было

Результат uptime	Причина	Что делать
up 1–2 min (плата только что перезагрузилась)	Скорее всего, просадка питания под нагрузкой	Заменить блок питания на 5 В / 4 А, взять короткий качественный кабель
up несколько минут и больше (как у нас)	Плата работала, оборвалось только SSH-соединение	Ничего страшного, продолжать

В нашем случае плата **не перезагружалась**. Сразу после первого запуска система сама начала ставить обновления безопасности в фоне (служба unattended-upgrades). Об этом говорит строка **«some packages require a reboot (libc6)»**: при обновлении системной библиотеки libc6 перезапускается SSH-сервер, и соединение рвётся. При этом сама команда `apt` на плате **продолжает работать**.

7.4. Ожидание блокировки apt

Если после переподключения запустить новую команду apt (например, `apt install -y tmux`), она будет ждать с сообщением **Waiting for cache lock … held by process … (apt)** (рис. 9).

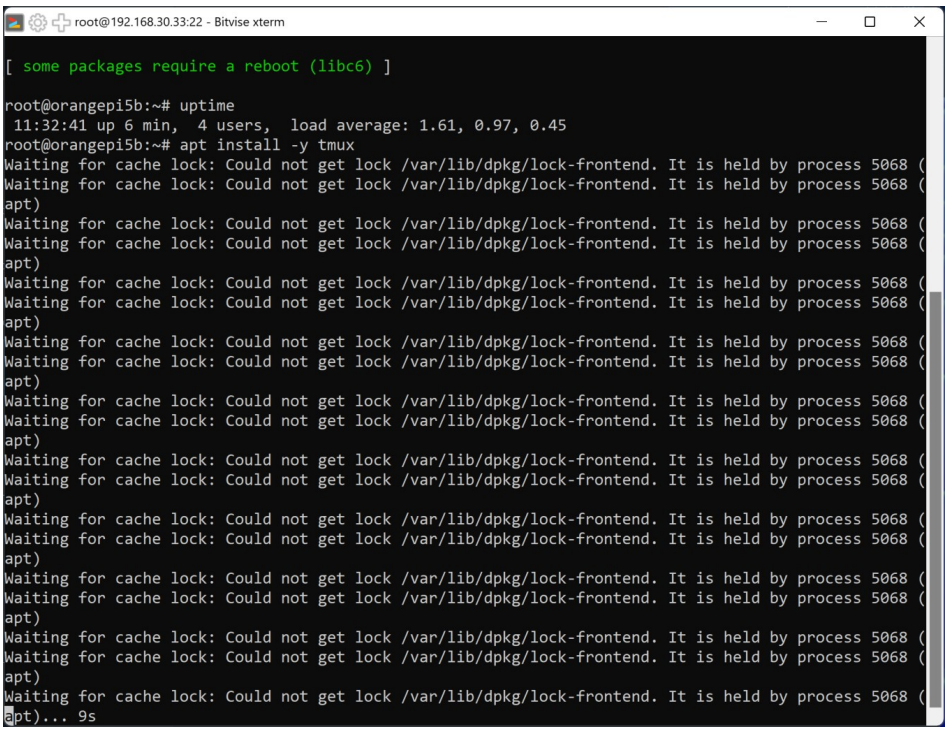


Рис. 10. Новая команда ждёт, пока завершится предыдущий процесс apt

Это нормально: предыдущий процесс apt продолжает устанавливать обновления. **Не прерывайте его** (не нажимайте Ctrl+C у старого процесса и не используйте `kill`) — иначе установка пакетов останется недоделанной. Новая команда выполнится автоматически, когда освободится блокировка.

Следить за ходом обновления можно во втором терминале (в Bitvise кнопка **New terminal console**):

```
tail -f /var/log/apt/term.log
```

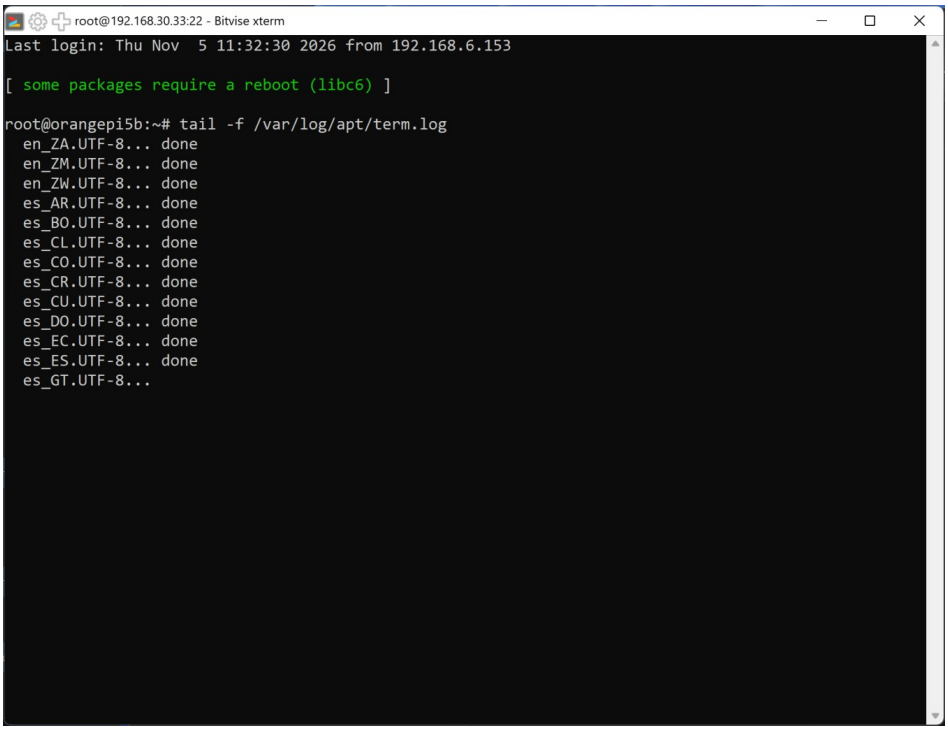


Рис. 11. Журнал apt: генерация языковых настроек (locales)

На рис. 10 видно, как система заново генерирует языковые настройки (`en_ZA`, `es_AR`, `es_GT` …) — их несколько сотен, поэтому этот шаг занимает 5–10 минут. Выйти из просмотра журнала — **Ctrl+C** (это останавливает только просмотр, а не обновление).

7.5. Завершение обновления

Обновление закончено, когда в журнале появится строка **Log ended** (рис. 12), а ожидавшая команда в первом окне выполнится и вернёт приглашение `root@orangepi5b:~#`.

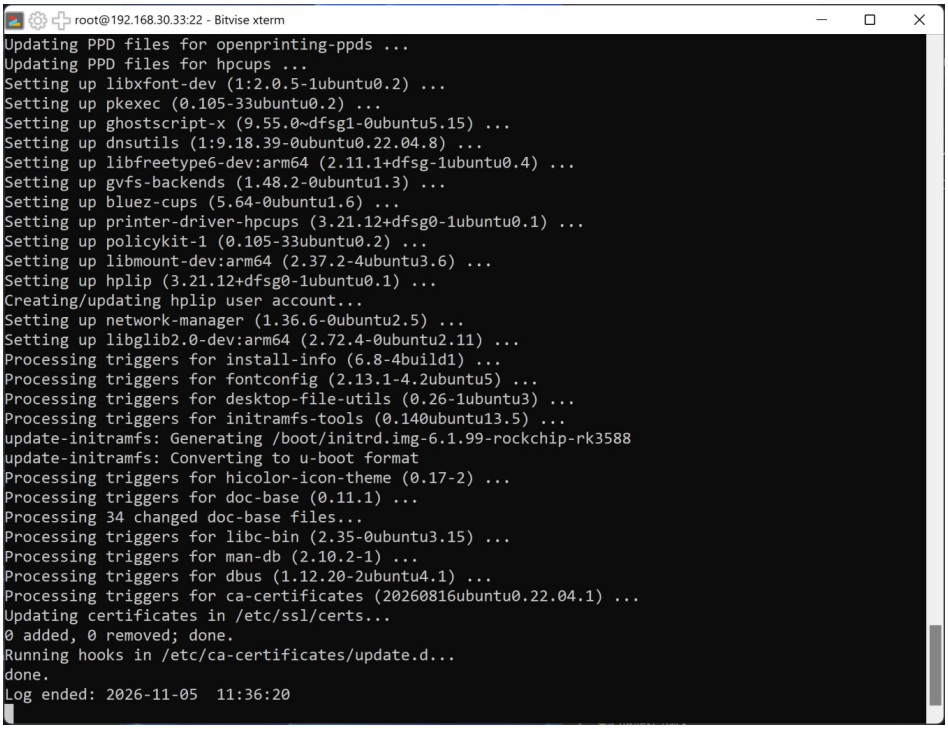


Рис. 12. Журнал apt: обновление завершено (Log ended)

Затем выполните:

```
dpkg --configure -a     # завершить настройку пакетов, если что-то осталось
reboot                  # перезагрузка (требуется после обновления libc6)
```

Сообщение **«44 not upgraded»** при последующих вызовах apt — нормально: это пакеты с поэтапным выпуском обновлений Ubuntu и пакеты ядра/загрузчика, которые Orange Pi намеренно не обновляет. Трогать их не нужно.

7.6. Как избежать обрывов в будущем

Длительные команды запускайте внутри **tmux** — тогда при обрыве SSH они продолжат работу на плате, а вы сможете вернуться к ним:

```
apt install -y tmux
tmux new -s main        # открыть сеанс (внизу появится зелёная строка [main])
apt update && apt upgrade -y
# ...если связь оборвалась — переподключиться и выполнить:
tmux attach -t main
```

Выйти из tmux: **Ctrl+B**, затем **D** — отсоединиться (сеанс продолжит работать); команда `exit` — закрыть сеанс совсем.

По желанию — ускорить будущие обновления, оставив только нужные языки:

```
dpkg-reconfigure locales
```

В списке оставьте отмеченными (клавиша пробел) только `en_US.UTF-8` и `ru_RU.UTF-8`.

8. Проверка питания под нагрузкой

RK3588 под нагрузкой потребляет заметный ток, а мотор лидара добавит ещё около 0,5 А. Если блок питания не справляется, плата внезапно перезагружается. Проверим это заранее нагрузочным тестом всех 8 ядер процессора на 1 минуту:

```
apt install -y stress-ng
uptime
stress-ng --cpu 8 --timeout 60s --metrics-brief
uptime
```

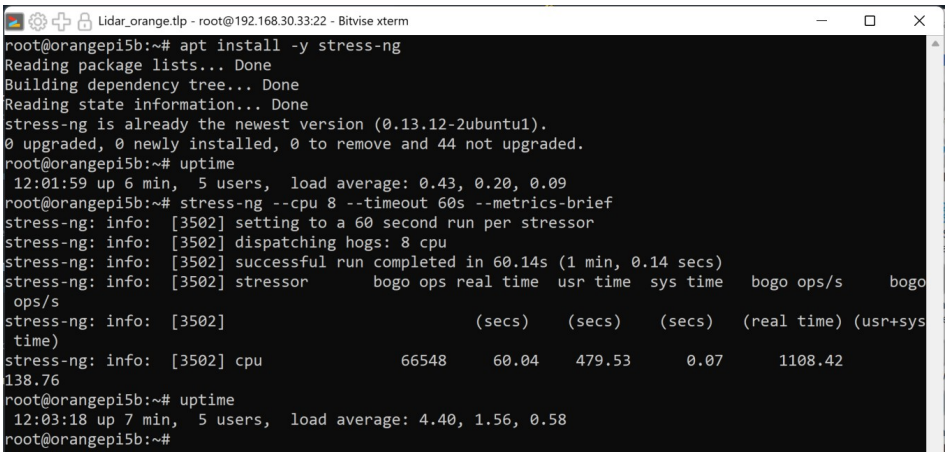


Рис. 13. Тест пройден: сводка stress-ng получена, время работы (uptime) выросло с 6 до 7 минут — перезагрузки не было

Результат	Вывод
Появилась сводка `successful run completed`, второй `uptime` больше первого на минуту	Питание выдерживает полную нагрузку процессора
Соединение оборвалось, после переподключения `uptime` показывает 0–1 мин	Плата перезагрузилась — нужен другой блок питания 5 В / 4 А и короткий качественный кабель

> **Как отличить перезагрузку от обрыва сети.** Команда `uptime` показывает, сколько плата работает без перезагрузки. Если значение большое — плата работала, а оборвалось только сетевое соединение.
>
> Журналы системы (`/var/log`) в этом образе хранятся в оперативной памяти (zram) и сохраняются на eMMC только при штатном выключении. Если после внезапной перезагрузки записи в журнале пропали — это признак пропадания питания.

9. Точка доступа Wi-Fi на плате

В нашей сети компьютер (подсеть 192.168.6.x) и плата (подсеть 192.168.30.x) оказались в разных подсетях, и маршрутизатор между ними периодически разрывал соединения SSH и удалённого рабочего стола. Самое надёжное решение — поднять на Orange Pi собственную точку доступа Wi-Fi и подключать компьютер к плате напрямую. Это же удобно и на роботе: плата всегда доступна по одному и тому же адресу без роутера.

> **Важно.** Точку доступа настраивайте, будучи подключённым к плате по Ethernet (или с монитором и клавиатурой), — тогда связь не пропадёт во время настройки.

9.1. Создание точки доступа

Проверьте, что Wi-Fi-адаптер свободен:

```
nmcli dev
```

В списке должна быть строка `wlan0  wifi  disconnected`. Создайте точку доступа (имя сети **OrangePi5B**, пароль **orangepi5b**):

```
nmcli dev wifi hotspot ifname wlan0 con-name OPi-Hotspot ssid OrangePi5B password
"orangepi5b"
nmcli con modify OPi-Hotspot 802-11-wireless-security.pmf disable
connection.autoconnect yes
nmcli con up OPi-Hotspot
ip -4 addr show wlan0
```

- `pmf disable` — отключает защиту служебных кадров (PMF): с ней Windows хуже подключается к этому Wi-Fi-модулю;
- `connection.autoconnect yes` — точка доступа будет подниматься автоматически после каждой перезагрузки;
- последняя команда должна показать адрес **10.42.0.1/24** — это адрес платы в её Wi-Fi-сети. Подключающимся устройствам плата сама раздаёт адреса 10.42.0.x.

9.2. Раздача интернета

NetworkManager в режиме точки доступа (`shared`) автоматически раздаёт подключённым устройствам интернет, который плата получает по Ethernet. Проверка:

```
nmcli -g ipv4.method con show OPi-Hotspot    # ожидается: shared
sysctl net.ipv4.ip_forward                   # ожидается: net.ipv4.ip_forward = 1
iptables -t nat -S | grep 10.42              # ожидается правило ... MASQUERADE
```

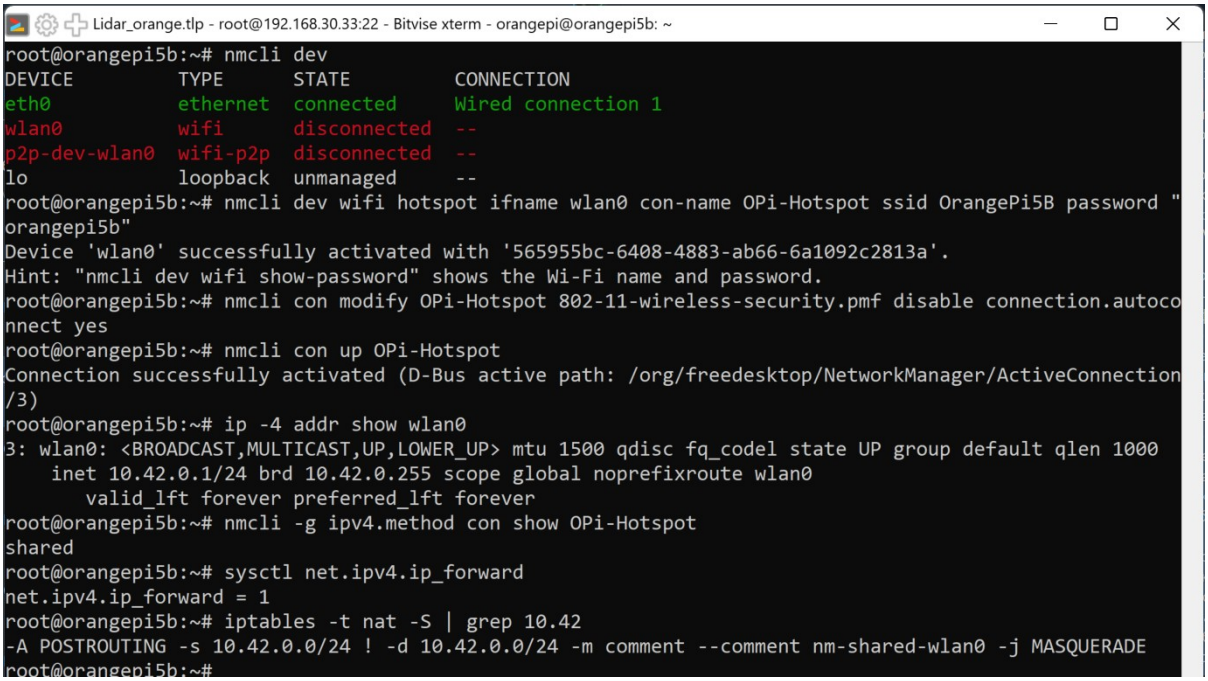


Рис. 14. Создание точки доступа и проверка раздачи интернета

9.3. Подключение компьютера

1. В списке сетей Wi-Fi Windows выберите **OrangePi5B**, пароль `orangepi5b`.
2. Проверьте связь в PowerShell: `ping 10.42.0.1` — плата должна отвечать; `ping 8.8.8.8` и `nslookup ya.ru` — должен работать интернет через плату.
3. В Bitvise создайте новый профиль с адресом **Host: 10.42.0.1** (Save profile as, например `Lidar_orange_wifi.tlp`). Логины и пароли — прежние.

Компьютер с MOC ОС подключается к той же сети Wi-Fi **OrangePi5B** (если MOC ОС запущена в VirtualBox на этом же ноутбуке с сетью NAT — отдельно ничего настраивать не нужно).

10. Удалённый рабочий стол (TigerVNC)

Для удалённого рабочего стола используем **TigerVNC** — бесплатный сервер из стандартных репозиториев Ubuntu. Он запускает отдельный рабочий стол Xfce, который работает и без подключённого монитора.

> **Почему не RDP и не NoMachine. xrdp** (RDP) из Ubuntu 22.04 (версия 0.9.17) несовместим с современными клиентами: и «Подключение к удалённому рабочему столу» Windows 11, и xfreerdp в MOC ОС зависали на этапе установления соединения (`Timeout waiting for activation`, ошибка 0x4).
>
> **NoMachine** версии 10 для ARM требует платную подписку и без неё не пускает («Подписка на этом сервере не существует»).

10.1. Установка

Под пользователем `root`:

```
apt install -y tigervnc-standalone-server tigervnc-common dbus-x11
```

10.2. Пароль VNC

Переключитесь на пользователя `orangepi` и задайте пароль для подключения (в примере — `orangepi`; символы при вводе не отображаются):

```
su - orangepi
vncpasswd
```

На вопрос **Would you like to enter a view-only password (y/n)?** ответьте `n` (пароль «только для просмотра» не нужен).

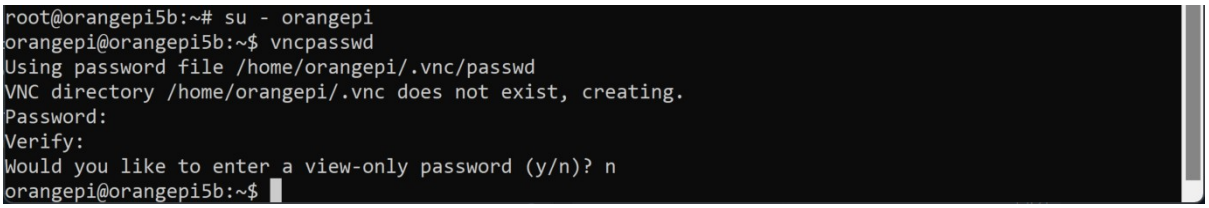


Рис. 15. Задание пароля VNC

10.3. Файл запуска рабочего стола

Оставаясь пользователем `orangepi`, создайте файл, который запускает Xfce внутри VNC, и проверьте запуск вручную:

```
mkdir -p ~/.vnc
cat > ~/.vnc/xstartup << 'EOF'
#!/bin/sh
unset SESSION_MANAGER
unset DBUS_SESSION_BUS_ADDRESS
export XDG_SESSION_TYPE=x11
exec dbus-launch --exit-with-session startxfce4
EOF
chmod +x ~/.vnc/xstartup
vncserver :1 -geometry 1600x900 -localhost no
```

Ожидаемый результат — строка `New Xtigervnc server 'orangepi5b:1 (orangepi)' on port 5901` (рис. 16). Параметры: `:1` — номер дисплея (порт 5901), `-geometry`

`1600x900` — разрешение рабочего стола, `-localhost no` — разрешить подключения по сети.

```
orangepi@orangepi5b:~$ vncserver -kill :1
Killing Xtigervnc process ID 20158... success!
orangepi@orangepi5b:~$ cat > ~/.vnc/xstartup << 'EOF'
> #!/bin/sh
> unset SESSION_MANAGER
> unset DBUS_SESSION_BUS_ADDRESS
> export XDG_SESSION_TYPE=x11
> exec dbus-launch --exit-with-session startxfce4
> EOF
orangepi@orangepi5b:~$ chmod +x ~/.vnc/xstartup
orangepi@orangepi5b:~$ vncserver :1 -geometry 1600x900 -localhost no

New Xtigervnc server 'orangepi5b:1 (orangepi)' on port 5901 for display :1.
Use xtigervncviewer -SecurityTypes VncAuth,TLSVnc -passwd /home/orangepi/.vnc/passwd orangepi5b:1 to c
onnect to the VNC server.

orangepi@orangepi5b:~$
```

Рис. 16. Файл xstartup с dbus-launch и запуск VNC-сервера

> **Зачем dbus-launch.** В официальном образе включён автовход: на основном экране платы уже открыт сеанс Xfce пользователя `orangepi`. Без отдельной шины D-Bus второй сеанс Xfce в VNC не запускается — клиент показывает чёрный экран. Строка `exec dbus-launch --exit-with-session startxfce4` даёт сеансу VNC собственную шину.

10.4. Автозапуск при загрузке

Чтобы VNC запускался сам после каждой перезагрузки, создайте службу systemd. Вернитесь в `root` командой `exit` и выполните:

```
cat > /etc/systemd/system/vncserver@.service << 'EOF'
[Unit]
Description=TigerVNC server on display :%i
After=network.target

[Service]
Type=forking
User=orangepi
WorkingDirectory=/home/orangepi
ExecStartPre=-/usr/bin/vncserver -kill :%i
ExecStart=/usr/bin/vncserver :%i -geometry 1600x900 -localhost no -SecurityTypes
VncAuth
ExecStop=/usr/bin/vncserver -kill :%i

[Install]
WantedBy=multi-user.target
EOF
su - orangepi -c "vncserver -kill :1"
systemctl daemon-reload
systemctl enable --now vncserver@1
systemctl status vncserver@1 --no-pager
```

Служба должна быть в состоянии **active (running)** (рис. 17). Красная строка `ExecStartPre … status=1/FAILURE` — не ошибка: это попытка остановить старый сервер, которого не было; знак `-` перед командой велит systemd её игнорировать.

```
root@orangepi5b:~# systemctl status vncserver@1 --no-pager
● vncserver@1.service - TigerVNC server on display :1
     Loaded: loaded (/etc/systemd/system/vncserver@.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-11-05 12:57:06 MSK; 3s ago
    Process: 21807 ExecStartPre=/usr/bin/vncserver -kill :1 (code=exited, status=1/FAILURE)
    Process: 21818 ExecStart=/usr/bin/vncserver :1 -geometry 1600x900 -localhost no (code=exited, stat
us=0/SUCCESS)
      Tasks: 110 (limit: 4515)
     Memory: 258.7M
        CPU: 9.010s
     CGroup: /system.slice/system-vncserver.slice/vncserver@1.service
             ├─21846 /usr/bin/perl /usr/bin/vncserver :1 -geometry 1600x900 -localhost no
```

Рис. 17. Служба vncserver@1 запущена, внутри работает рабочий стол Xfce

> **Параметр -SecurityTypes VncAuth** разрешает простую проверку пароля VNC. По умолчанию TigerVNC предлагает только шифрованный вариант (VeNCrypt), который не поддерживает клиент KRDC в МОС ОС (ошибка `rfbInitClient failed`). Сеть платы и так защищена паролем Wi-Fi (WPA2). Если служба уже была создана без этого параметра, добавить его можно так:

```
sed -i 's|-localhost no$|-localhost no -SecurityTypes VncAuth|'
/etc/systemd/system/vncserver@.service
systemctl daemon-reload
systemctl restart vncserver@1
```

```
root@orangepi5b:~# sed -i 's|-localhost no$|-localhost no -SecurityTypes VncAuth|' /etc/systemd/system
/vncserver@.service
root@orangepi5b:~# grep ExecStart= /etc/systemd/system/vncserver@.service
ExecStart=/usr/bin/vncserver :%i -geometry 1600x900 -localhost no -SecurityTypes VncAuth
root@orangepi5b:~# systemctl daemon-reload
root@orangepi5b:~# systemctl restart vncserver@1
root@orangepi5b:~# systemctl is-active vncserver@1
active
root@orangepi5b:~#
```

Рис. 18. Добавление -SecurityTypes VncAuth и перезапуск службы

11. Подключение к рабочему столу

11.1. С Windows — TigerVNC Viewer

1. Скачайте **TigerVNC Viewer**: github.com/TigerVNC/tigervnc → Releases → ссылка на SourceForge → файл `vncviewer64-1.x.x.exe`. Установка не нужна — это один исполняемый файл.
2. Запустите его, в поле **VNC server** введите `10.42.0.1:1` (рис. 19) и нажмите **Connect**.
3. Введите пароль VNC `orangepi`.
4. Подключение можно сохранить кнопкой **Save as…** и потом открывать через **Load…**. Клавиша **F8** внутри окна открывает меню (полный экран, отключение).



Рис. 19. Окно TigerVNC Viewer

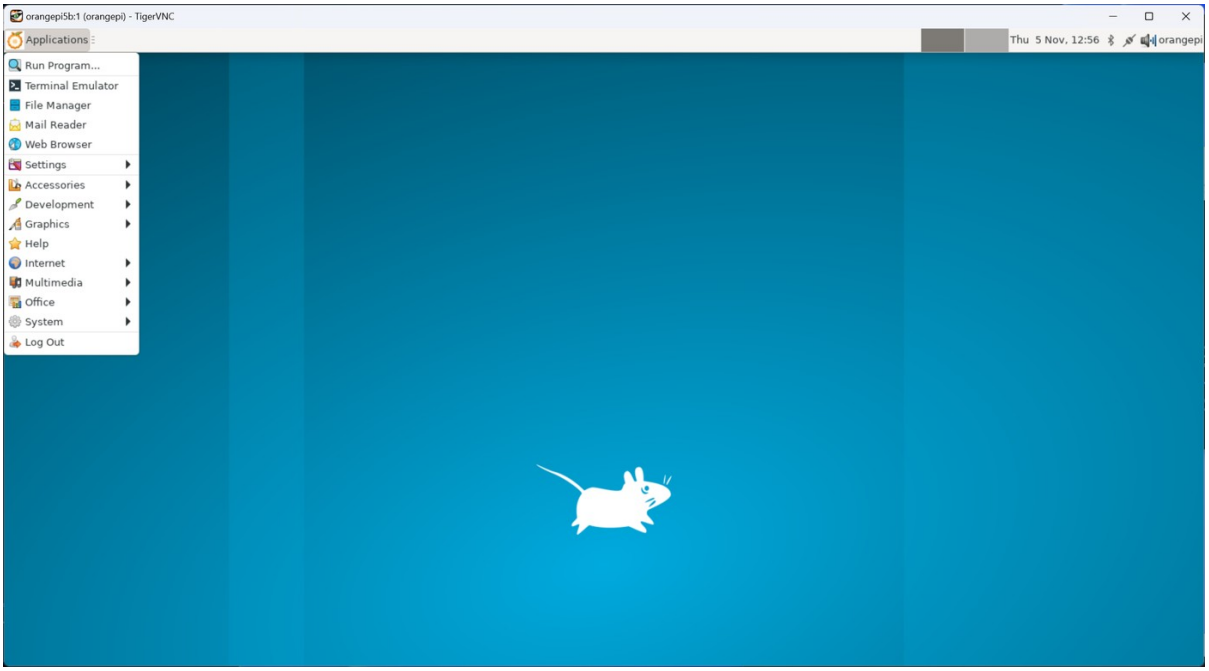


Рис. 20. Рабочий стол Xfce платы в TigerVNC Viewer (Windows)

11.2. С МОС ОС — KRDC

В МОС ОС уже установлен **KRDC** — клиент удалённого рабочего стола KDE. Сначала проверьте связь с платой в терминале (Konsole):

```
ping -c 3 10.42.0.1
timeout 3 bash -c '</dev/tcp/10.42.0.1/5901' && echo "VNC ОТКРЫТ" || echo "VNC ЗАКРЫТ"
```

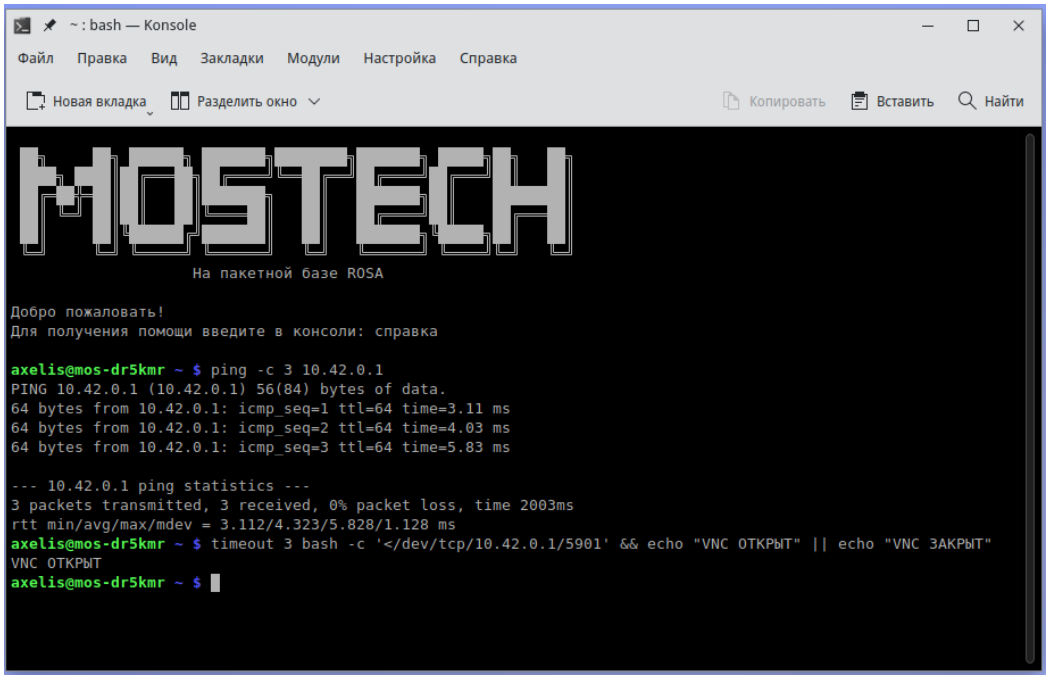


Рис. 21. МОС ОС: плата отвечает, порт VNC 5901 открыт

Запустите KRDC сразу с адресом:

```
krdc vnc://10.42.0.1:5901 &
```

Порт указывается полностью (`5901`) — в KRDC запись `:1` может быть воспринята как порт 1. В окне **Параметры хоста** выберите **Тип соединения: Высокое качество (локальная сеть, прямое соединение)** (рис. 22), нажмите **OK** и введите пароль `orangepi`.

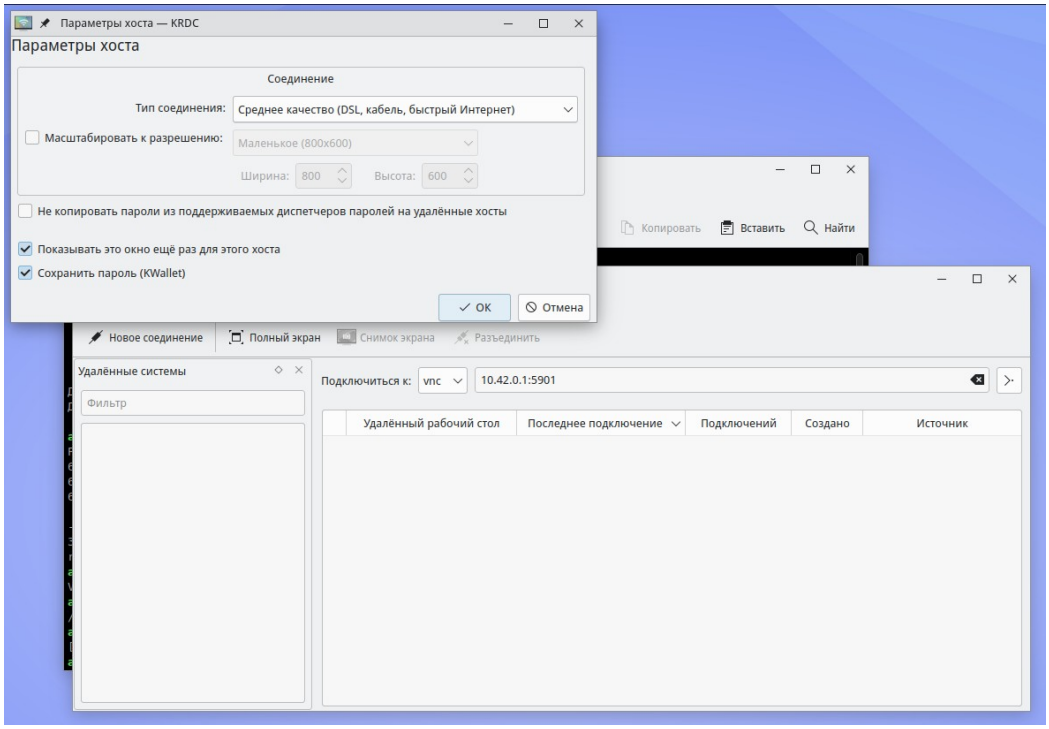


Рис. 22. KRDC: параметры подключения

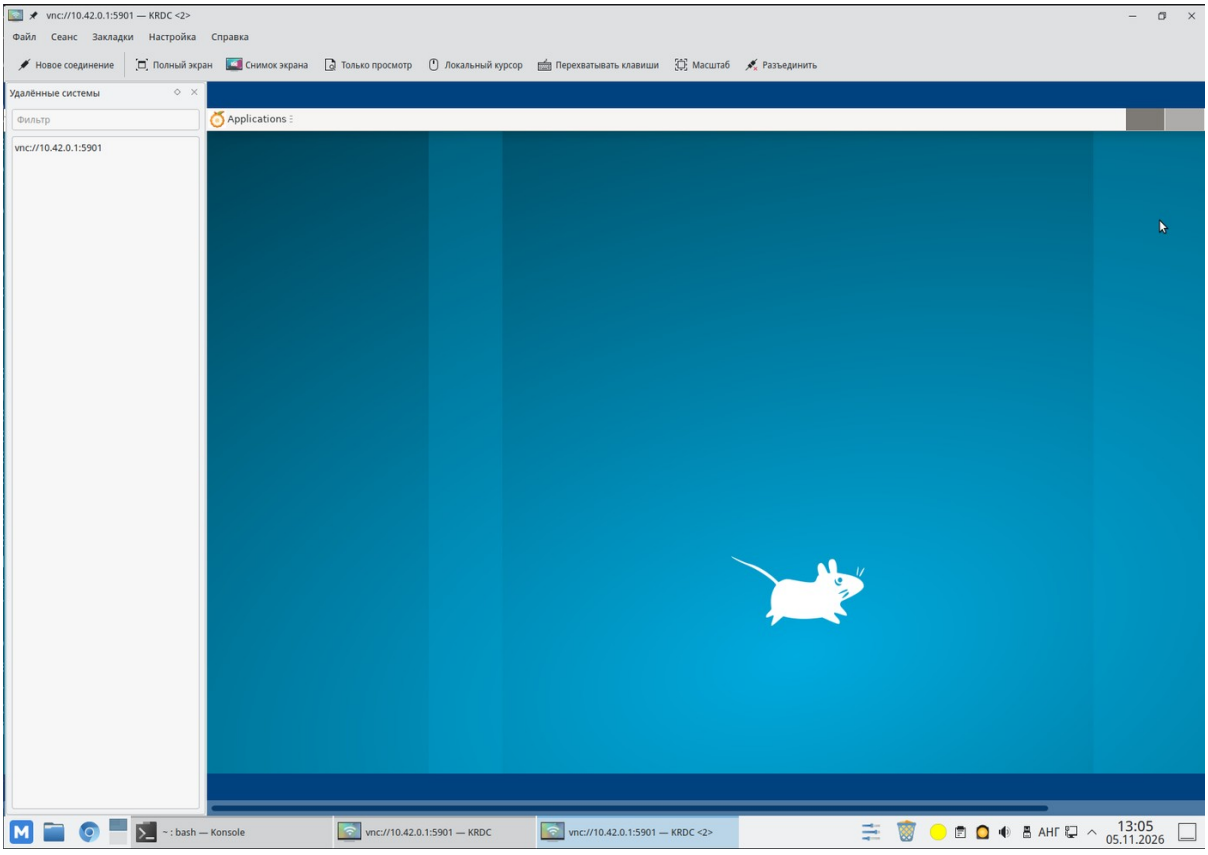


Рис. 23. Рабочий стол платы в KRDC (MOC ОС)

Кнопки KRDC: **Масштаб** — подогнать рабочий стол под размер окна, **Полный экран** — развернуть на весь монитор.

11.3. С МОС ОС — терминал (SSH) и файлы (SFTP)

Программа Bitvise SSH Client существует только для Windows. В МОС ОС всё необходимое уже встроено: команда `ssh` в терминале **Konsole** и поддержка SFTP в файловом менеджере **Dolphin**.

Подключение по SSH. В Konsole:

```
ssh orangepi@10.42.0.1
```

Пароль `orangepi`. При первом подключении на вопрос о ключе сервера ответьте `yes`.

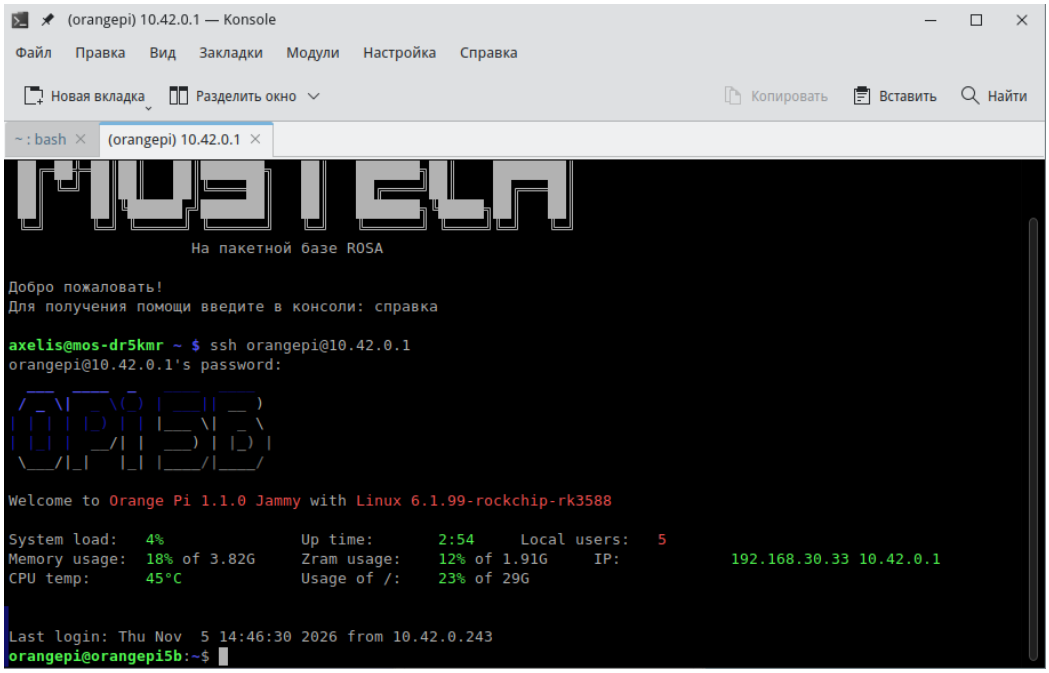


Рис. 24. МОС ОС: подключение к плате по SSH в Konsole

Профиль подключения (аналог файла .tlp в Bitvise) — чтобы не набирать адрес каждый раз:

```
mkdir -p ~/.ssh
cat >> ~/.ssh/config << 'EOF'
Host opi
    HostName 10.42.0.1
    User orangepi
    ServerAliveInterval 30
EOF
```

Теперь подключение — одной командой `ssh opi`. Строка `ServerAliveInterval 30` раз в 30 секунд отправляет служебный сигнал, чтобы соединение не обрывалось при паузах в работе.

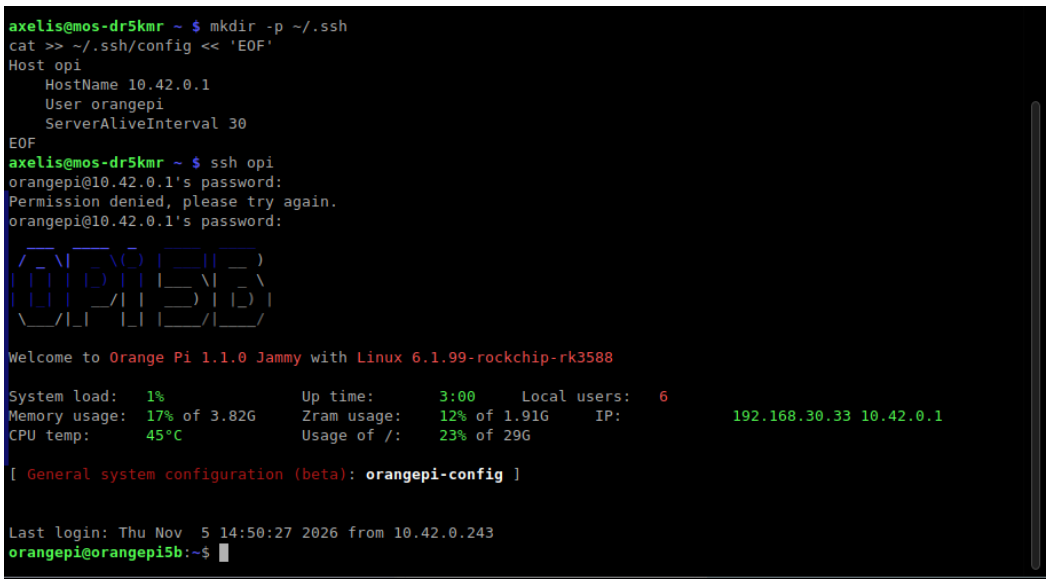


Рис. 25. Создание профиля opi и подключение командой ssh opi

Вход без пароля (по желанию) — по ключу:

```
ssh-keygen -t ed25519      # на все вопросы нажать Enter
ssh-copy-id opi            # один раз ввести пароль orangepi
```

Второй терминал (аналог кнопки New terminal console в Bitvise): в Konsole откройте новую вкладку (**Ctrl+Shift+T**) и снова выполните `ssh opi`.

Обмен файлами в Dolphin. Откройте Dolphin, нажмите **Ctrl+L** и введите адрес:

```
sftp://orangepi@10.42.0.1/home/orangepi
```

> **Внимание:** это адрес для строки адреса Dolphin, а не команда терминала. Если ввести его в Konsole, будет ошибка «Нет такого файла или каталога». Открыть Dolphin сразу с этим адресом из терминала можно командой `dolphin sftp://orangepi@10.42.0.1/home/orangepi &`.

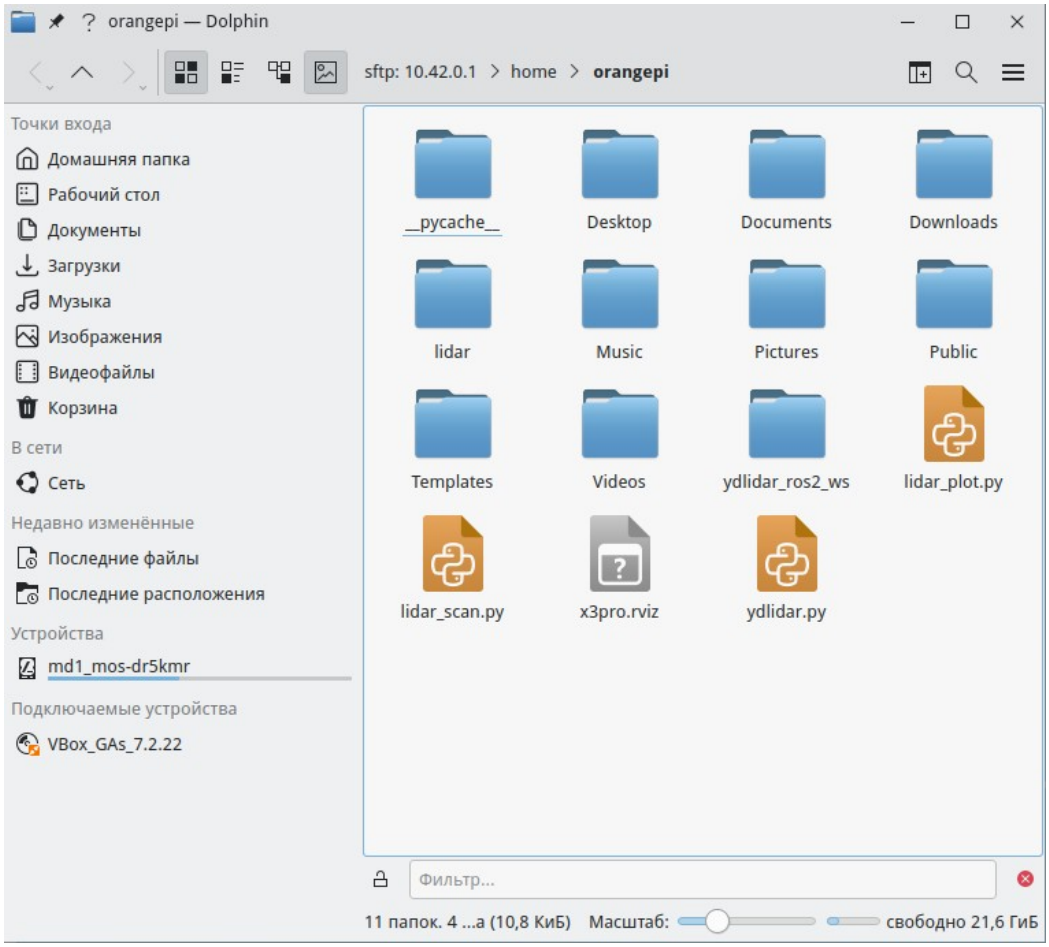


Рис. 26. Dolphin в MOC ОС: домашняя папка платы по SFTP

Домашняя папка платы откроется как обычная папка — файлы можно перетаскивать мышкой между компьютером и платой, как в окне SFTP у Bitvise. Чтобы адрес был всегда под рукой: правой кнопкой мыши по папке → **Добавить в «Точки входа»**.

Обмен файлами из терминала (используется профиль `opi`):

```
scp ydlidar.py lidar_plot.py opi:~/     # скопировать программы на плату
scp opi:~/room_map.pgm opi:~/room_map.yaml .   # забрать карту с платы
```

Bitvise (Windows)	MOC ОС
Окно терминала	Konsole: `ssh opi`
Профиль .tlp	Файл `~/.ssh/config` (раздел Host opi)
New terminal console	Новая вкладка Konsole (Ctrl+Shift+T) + `ssh opi`
Окно SFTP	Dolphin: `sftp://orangepi@10.42.0.1/home/orangepi`; в терминале — `scp`
Удалённый рабочий стол	KRDC: `vnc://10.42.0.1:5901` (раздел 11.2)

12. Сводка: как подключаться к плате

Что	Адрес / параметры	Логин / пароль
Wi-Fi платы	сеть `OrangePi5B`	пароль `orangepi5b`

Что	Адрес / параметры	Логин / пароль
SSH (Bitvise на Windows, `ssh opi` на МОС ОС)	`10.42.0.1`, порт 22	`orangepi` / `orangepi` или `root` / `orangepi`
VNC с Windows (TigerVNC Viewer)	`10.42.0.1:1`	пароль VNC `orangepi`
VNC с МОС ОС (KRDC)	`vnc://10.42.0.1:5901`	пароль VNC `orangepi`
Файлы с МОС ОС (Dolphin)	`sftp://orangepi@10.42.0.1/home/orangepi`	`orangepi` / `orangepi`
Через общую сеть (Ethernet)	IP из роутера (у нас `192.168.30.33`)	те же

> **Безопасность.** Пароли по умолчанию допустимы только в учебной локальной сети. Не открывайте доступ к плате из интернета (не пробрасывайте порты 22 и 5901 на роутере).

13. Подключение лидара YDLIDAR X3 Pro

13.1. Подготовка системы

Под пользователем `root`:

```
usermod -aG dialout orangepi          # доступ к последовательным портам
apt remove -y brltty                  # служба, перехватывающая USB-UART адаптеры
apt install -y python3-venv python3-pip python3-tk
```

13.2. Подключение и проверка порта

1. Вставьте шлейф лидара в плату-адаптер из комплекта.
2. Подключите адаптер кабелем USB Type-C к любому USB-порту Orange Pi. Головка лидара сразу начинает вращаться — мотор управляется встроенным контроллером.
3. Проверьте, что система увидела адаптер:

```
ls /dev/ttyUSB*
dmesg | tail -15
```

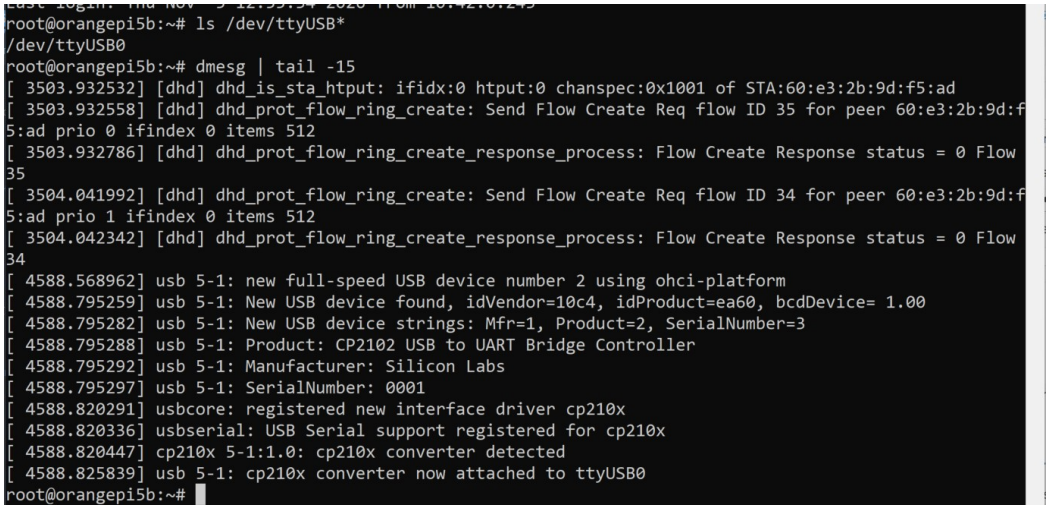


Рис. 27. Адаптер лидара определён: CP2102 USB to UART (Silicon Labs), порт /dev/ttyUSB0

Адаптер собран на микросхеме **CP2102**, драйвер `cp210x` встроен в систему и загружается автоматически. Порт лидара — **/dev/ttyUSB0**, скорость **115200 бод**.

13.3. Окружение Python

Дальше работаем под обычным пользователем `orangepi`. Убедитесь, что он входит в группу `dialout`, и создайте виртуальное окружение Python с нужными библиотеками:

```
su - orangepi
groups                          # в списке должно быть dialout
python3 -m venv ~/lidar
source ~/lidar/bin/activate     # приглашение начнётся с (lidar)
pip install pyserial matplotlib
```

В каждом новом сеансе окружение нужно активировать заново командой `source ~/lidar/bin/activate`.

13.4. Проверка потока данных

Лидар X3 Pro передаёт данные непрерывно, без команд запуска. Прочитаем 2000 байт и посчитаем пакеты:

```
python3 - << 'EOF'
import serial
s = serial.Serial('/dev/ttyUSB0', 115200, timeout=2)
data = s.read(2000)
s.close()
print('Получено байт:', len(data))
print('Пакетов с заголовком AA 55:', data.count(b'\xaa\x55'))
print(data[:40].hex(' '))
EOF
```

```
(lidar) orangepi@orangepi5b:~$ python3 - << 'EOF'
> import serial
> s = serial.Serial('/dev/ttyUSB0', 115200, timeout=2)
> data = s.read(2000)
> s.close()
> print('Получено байт:', len(data))
> print('Пакетов с заголовком AA 55:', data.count(b'\xaa\x55'))
> print(data[:40].hex(' '))
> EOF
Получено байт: 2000
Пакетов с заголовком AA 55: 34
aa 55 62 28 4b 3e bd 47 2c 31 a4 17 b0 17 c0 17 d0 17 e0 17 fc 17 00 18 20 18 48 18 00 00 e2 1a e8 1a
f4 1a 00 1b 10 1b
(lidar) orangepi@orangepi5b:~$
```

Рис. 28. Данные идут: 2000 байт, 34 заголовка пакетов AA 55

Формат пакета YDLIDAR. Расшифруем начало первого пакета с рис. 25:

Байты	Поле	Значение
`aa 55`	Заголовок (PH)	Начало пакета
`62`	Тип пакета (CT)	Младший бит 0 — обычный пакет; 1 — начало нового оборота
`28`	Число точек (LSN)	0x28 = 40 точек
`4b 3e`	Начальный угол (FSA)	0x3E4B → (0x3E4B >> 1) / 64 = 124,6°
`bd 47`	Конечный угол (LSA)	0x47BD → (0x47BD >> 1) / 64 = 143,5°
`2c 31`	Контрольная сумма (CS)	XOR всех 16-битных слов пакета
`a4 17`, `b0 17` …	Расстояния (по 2 байта на точку)	0x17A4 = 6052 → 6052 / 4 = 1513 мм; 0 — нет отражения

Числа передаются в порядке «младший байт первым» (`4b 3e` = 0x3E4B). Углы промежуточных точек вычисляются равномерно между FSA и LSA.

13.5. Модуль разбора данных

Функции разбора вынесены в отдельный модуль `ydlidar.py`, чтобы использовать их в разных программах. Создайте файл (или скопируйте готовый файл из папки с инструкцией в домашнюю папку `/home/orangepi` через окно SFTP в Bitvise):

```
cat > ~/ydlidar.py << 'EOF'
"""Разбор потока данных лидара YDLIDAR X3 Pro."""
import struct

def packets(ser):
    """Выделяет из потока пакеты и проверяет контрольную сумму."""
    buf = bytearray()
    while True:
        buf += ser.read(512)
        while True:
            i = buf.find(b'\xaa\x55')
            if i < 0:
                del buf[:-1]; break
            if len(buf) < i + 10:
                del buf[:i]; break
            ct, lsn = buf[i + 2], buf[i + 3]
            n = 10 + 2 * lsn
            if len(buf) < i + n:
                del buf[:i]; break
            pkt = bytes(buf[i:i + n])
            fsa, lsa, cs = struct.unpack_from('<HHH', pkt, 4)
            samples = struct.unpack_from('<%dH' % lsn, pkt, 10)
            chk = 0x55AA ^ (ct | lsn << 8) ^ fsa ^ lsa
            for s in samples:
                chk ^= s
            if chk != cs:            # повреждённый пакет — пропускаем
                del buf[:i + 2]; continue
            del buf[:i + n]
            yield ct, fsa, lsa, samples

def scans(ser, min_mm=100):
    """Возвращает списки точек (угол в градусах, расстояние в мм) за каждый
оборот."""
    pts = []
    for ct, fsa, lsa, samples in packets(ser):
        if ct & 1:                   # пакет начала нового оборота
            if pts:
                yield pts
            pts = []
        a1, a2 = (fsa >> 1) / 64, (lsa >> 1) / 64
        n = len(samples)
        diff = (a2 - a1) % 360
        for k, s in enumerate(samples):
            ang = (a1 + (diff * k / (n - 1) if n > 1 else 0)) % 360
            d = s / 4                # миллиметры
            if d >= min_mm:          # ближе 0,1 м лидар не измеряет
                pts.append((ang, d))
EOF
```

- `packets()` — находит в потоке заголовок `AA 55`, вырезает пакет нужной длины и проверяет контрольную сумму; повреждённые пакеты пропускаются;
- `scans()` — переводит пакеты в точки (угол, расстояние) и собирает их в полный оборот; новый оборот начинается с пакета, у которого установлен младший бит CT.

13.6. Текстовый вывод скана

```
cat > ~/lidar_scan.py << 'EOF'
"""Текстовый вывод скана: ближайшее препятствие и минимумы по 8 секторам."""
import time
import serial
from ydlidar import scans

ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
ser.reset_input_buffer()
print('Ctrl+C — остановить')
t0 = None
try:
    for i, pts in enumerate(scans(ser)):
        t = time.time()
        if i == 0 or not pts:
            t0 = t; continue
        hz = 1 / (t - t0); t0 = t
        ang, d = min(pts, key=lambda p: p[1])
        sect = [min([p[1] for p in pts if int(p[0] // 45) == k] or [0]) for k in
range(8)]
        print(f'{len(pts):4d} точек {hz:4.1f} Гц | ближайшее {d/1000:4.2f} м на
{ang:5.1f}° | '
              'секторы по 45° (м): ' + ' '.join(f'{x/1000:4.2f}' for x in sect))
except KeyboardInterrupt:
    pass
finally:
    ser.close()
EOF
python3 ~/lidar_scan.py
```

Для каждого оборота программа выводит число точек, частоту, расстояние до ближайшего препятствия с углом и минимальное расстояние в каждом из 8 секторов по 45° (рис. 26). Поднесите руку к лидару — значение «ближайшее» изменится. Остановка — **Ctrl+C**.

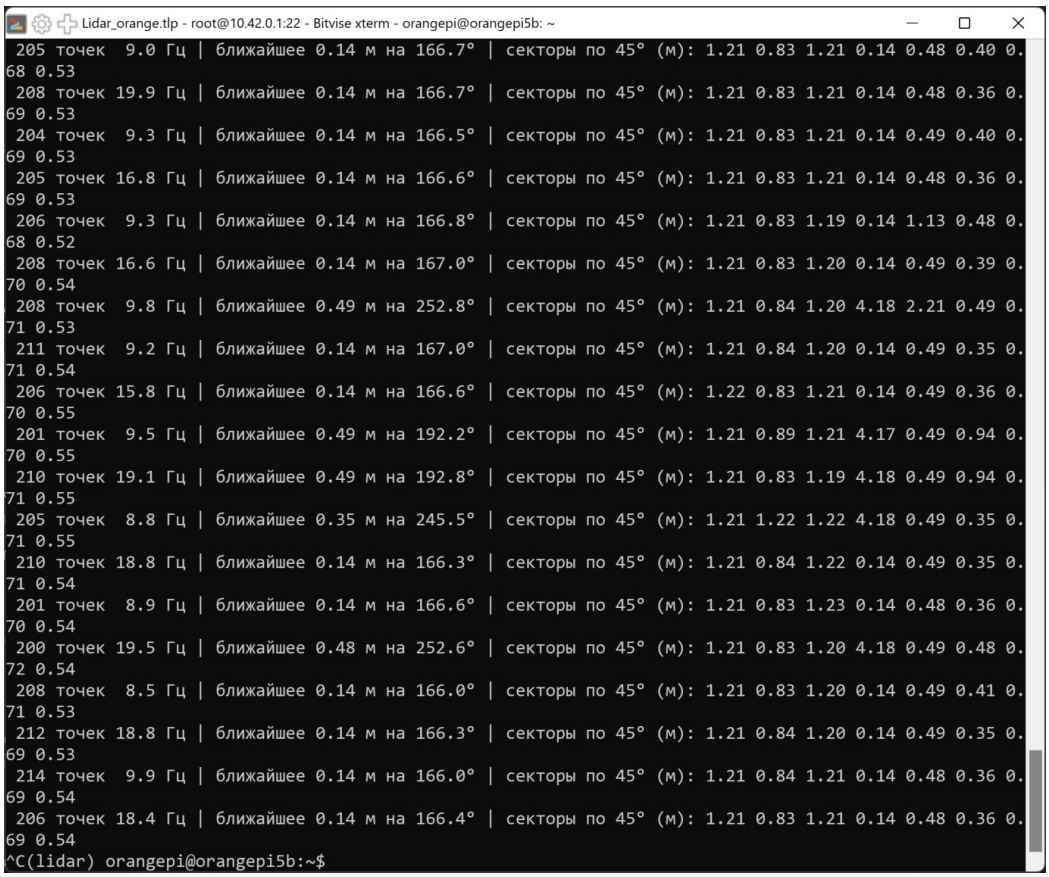


Рис. 29. Текстовый вывод: стабильные расстояния по секторам

> **Почему частота в текстовом выводе «скачет» (9–19 Гц).** Программа читает порт порциями по 512 байт (около 44 мс), поэтому время между оборотами измеряется грубо. Частоту вращения точнее всего показывает драйвер ROS 2 (раздел 14.5): **5,3 об/с**. Значения частоты в наших простых программах — ориентировочные.

13.7. Графическое отображение скана

```
cat > ~/lidar_plot.py << 'EOF'
"""Графическое отображение скана лидара (круговая диаграмма)."""
import sys, math, time, threading, collections
import serial
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from ydlidar import scans

RMAX = float(sys.argv[1]) if len(sys.argv) > 1 else 3.0   # масштаб, м

ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
ser.reset_input_buffer()
state = {'pts': [], 'times': collections.deque(maxlen=20)}

def reader():                         # чтение лидара в отдельном потоке
    for pts in scans(ser):
        state['pts'] = pts
        state['times'].append(time.time())
threading.Thread(target=reader, daemon=True).start()

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection='polar')
ax.set_theta_zero_location('N')     # 0° — вверх
ax.set_theta_direction(-1)          # углы по часовой стрелке
ax.set_rlim(0, RMAX)
sc = ax.scatter([], [], s=5, c='tab:red')
title = ax.set_title('')

def update(_):
    pts = state['pts']
    if pts:
        sc.set_offsets([(math.radians(a), d / 1000) for a, d in pts])
        t = state['times']
        hz = (len(t) - 1) / (t[-1] - t[0]) if len(t) > 1 and t[-1] > t[0] else 0
        title.set_text(f'YDLIDAR X3 Pro: {len(pts)} точек, {hz:.1f} об/с')
    return sc, title

ani = FuncAnimation(fig, update, interval=100, cache_frame_data=False)
plt.show()
ser.close()
EOF
```

Откройте рабочий стол платы по VNC (раздел 11) и запустите программу из Bitvise, указав дисплей VNC:

```
DISPLAY=:1 python3 ~/lidar_plot.py        # масштаб по умолчанию 3 м
DISPLAY=:1 python3 ~/lidar_plot.py 6      # масштаб 6 м
```

Можно запустить её и в терминале на самом рабочем столе VNC (Applications → Terminal Emulator), тогда `DISPLAY=:1` указывать не нужно. На круговой диаграмме 0° — вверху, углы растут по часовой стрелке, окружности сетки — расстояние в метрах. В заголовке — число точек и частота вращения.

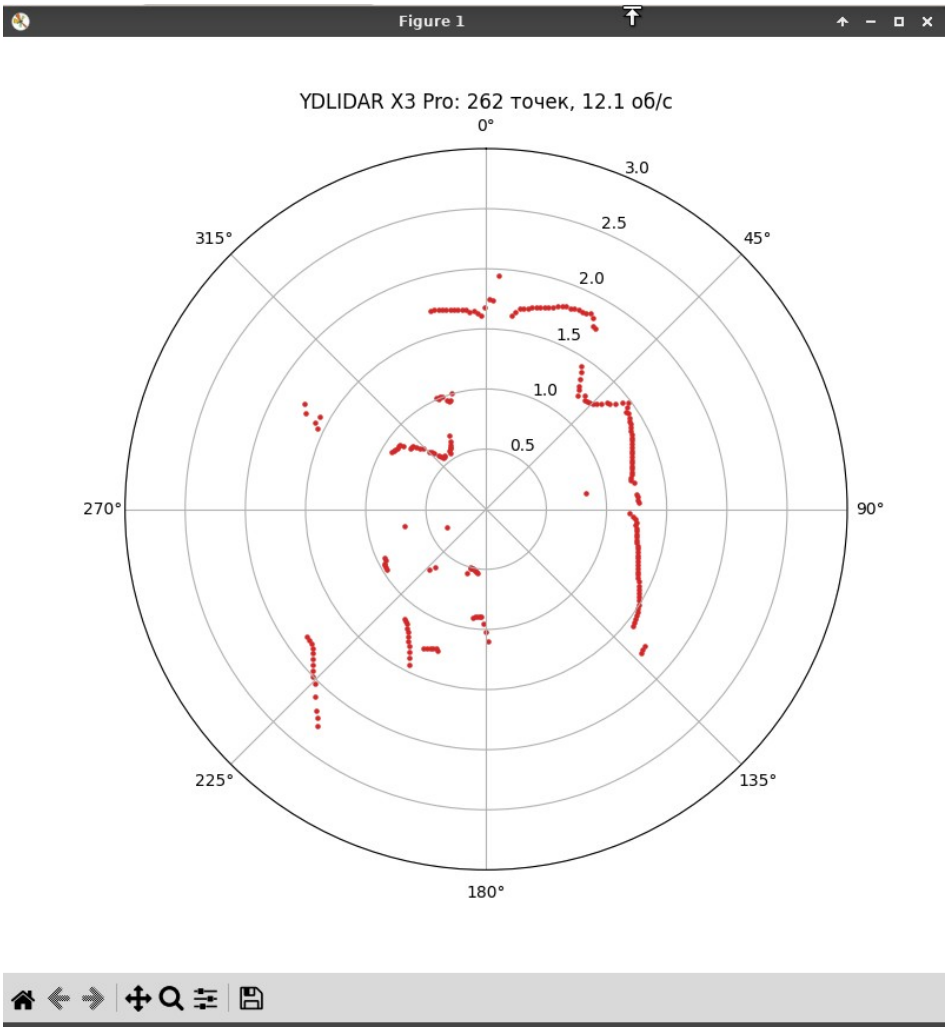


Рис. 30. Скан комнаты: справа стена на расстоянии ~0,8–1 м, впереди ~1,7 м

Окно обновляется 10 раз в секунду. Чтение лидара идёт в отдельном потоке (`threading`), поэтому отрисовка не мешает приёму данных. Закрыть программу — закрыть окно или Ctrl+C в терминале.

> **Результат проверки.** Лидар выдаёт около 4500 измерений в секунду. Расстояния стабильны от оборота к обороту, контуры стен на диаграмме прямые — лидар готов к использованию в проектах (обход препятствий, построение карты).

14. ROS 2 и визуализация в RViz2

ROS 2 (Robot Operating System) — стандартная программная платформа для роботов. Для лидара YDLIDAR есть официальный драйвер ROS 2: он публикует скан в топик `/scan`, а программа **RViz2** отображает его и любые другие данные робота. На Ubuntu 22.04 устанавливается версия ROS 2 **Humble**.

> **Перед началом** закройте `lidar_plot.py` и `lidar_scan.py`: последовательный порт лидара может использовать только одна программа. Длительные установки выполняйте в tmux (раздел 7.6).

14.1. Установка ROS 2 Humble

Под пользователем `root`. Языковые настройки и репозиторий universe:

```
tmux new -s ros
locale-gen en_US en_US.UTF-8
update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
apt install -y software-properties-common curl
add-apt-repository -y universe
```

Подключение репозитория ROS 2 (пакет `ros2-apt-source` сам добавляет адрес и ключ репозитория):

```
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-
infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F'"'
'{print $4}')
echo "Версия: $ROS_APT_SOURCE_VERSION"
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-
apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_$
{ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo $UBUNTU_CODENAME)_all.deb"
dpkg -i /tmp/ros2-apt-source.deb
```

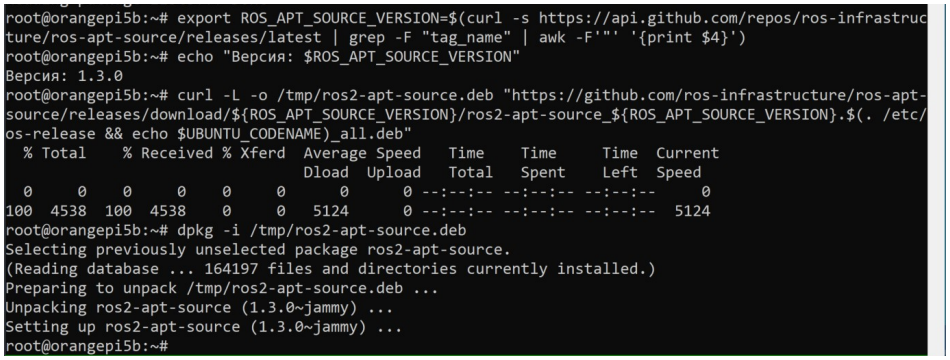


Рис. 31. Репозиторий ROS 2 подключён: ros2-apt-source 1.3.0~jammy

Установка базового набора ROS 2, RViz2 и инструментов сборки (10–20 минут). Полный набор `ros-humble-desktop` (несколько гигабайт) не нужен:

```
apt update
apt install -y ros-humble-ros-base ros-humble-rviz2 ros-humble-tf2-ros ros-dev-
tools cmake git pkg-config swig python3-dev
```

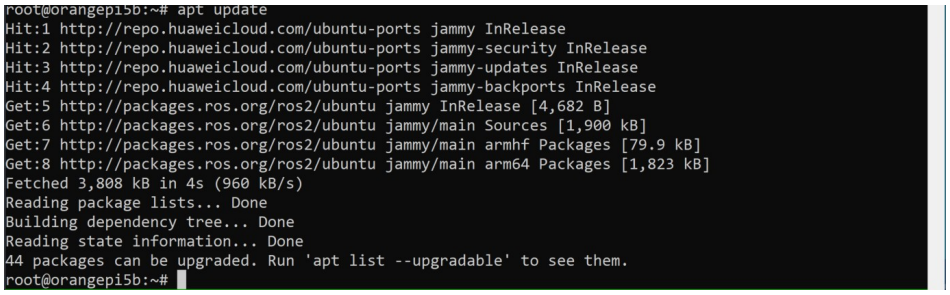


Рис. 32. apt update: загружены списки пакетов с packages.ros.org

Проверка:

```
source /opt/ros/humble/setup.bash
ros2 --help | head -3
```

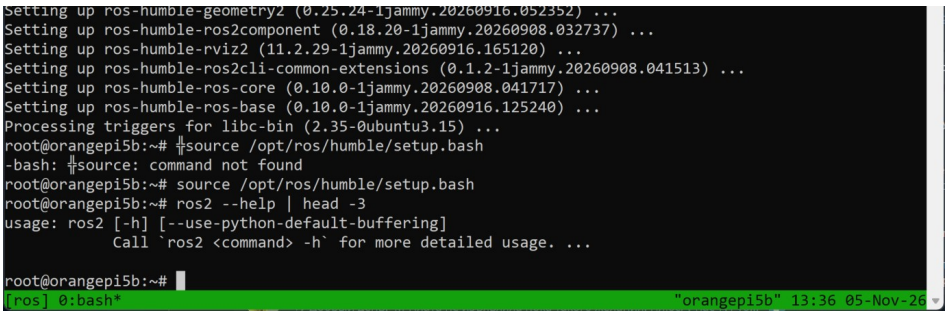


Рис. 33. ROS 2 Humble и RViz2 установлены, команда ros2 работает

14.2. Библиотека YDLidar-SDK

Библиотека производителя, через которую работает драйвер ROS 2. Под `root`:

```
cd /usr/local/src
git clone https://github.com/YDLIDAR/YDLidar-SDK.git
mkdir -p YDLidar-SDK/build && cd YDLidar-SDK/build
cmake ..
make -j4
make install
ldconfig
cd ~
```

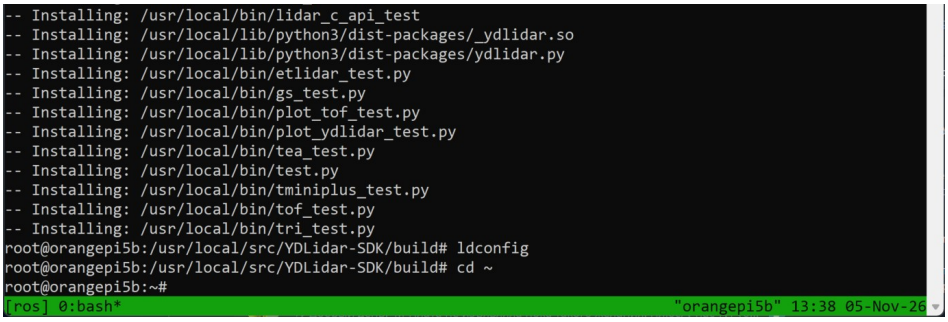


Рис. 34. YDLidar-SDK установлен в /usr/local (библиотека, тестовые программы, модуль Python)

14.3. Драйвер ydlidar_ros2_driver

Драйвер собирается в рабочей области (workspace) пользователя `orangepi`:

```
su - orangepi
source /opt/ros/humble/setup.bash
mkdir -p ~/ydlidar_ros2_ws/src && cd ~/ydlidar_ros2_ws/src
git clone -b humble https://github.com/YDLIDAR/ydlidar_ros2_driver.git || git
clone https://github.com/YDLIDAR/ydlidar_ros2_driver.git
cd ~/ydlidar_ros2_ws
colcon build --symlink-install
```

Сборка занимает около минуты и заканчивается строкой `Summary: 1 package finished`. Предупреждения `unused parameter` — особенность кода драйвера, на работу они не влияют.

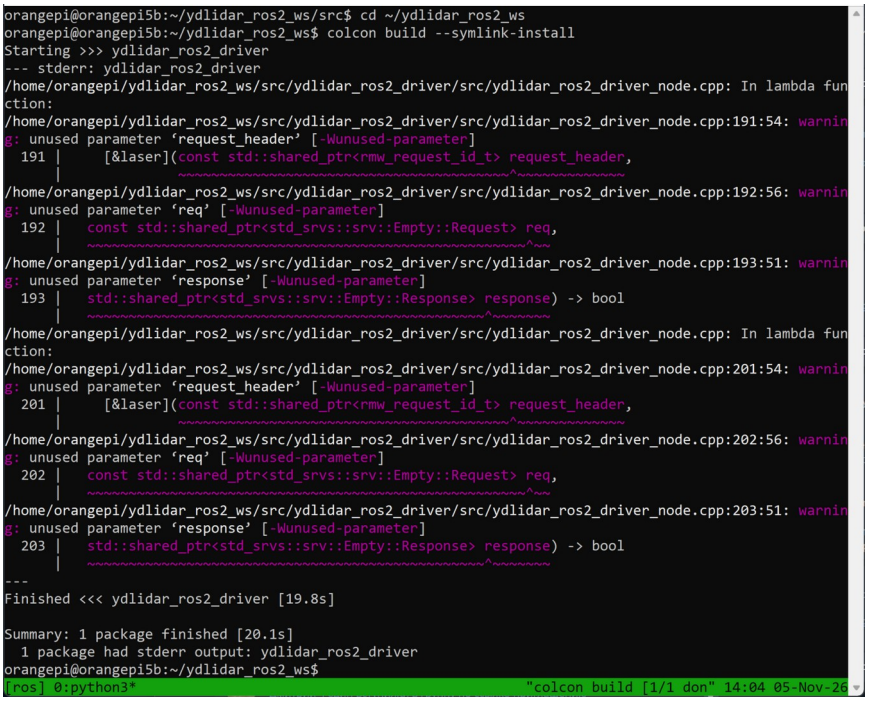


Рис. 35. Драйвер собран: Summary: 1 package finished

Чтобы ROS 2 и драйвер подключались автоматически в каждом новом терминале:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
echo "source ~/ydlidar_ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

14.4. Параметры лидара X3 Pro

Файл параметров драйвера `~/ydlidar_ros2_ws/x3pro.yaml` (готовый файл лежит в папке с инструкцией):

```
cat > ~/ydlidar_ros2_ws/x3pro.yaml << 'EOF'
ydlidar_ros2_driver_node:
  ros__parameters:
    port: /dev/ttyUSB0
    frame_id: laser_frame
    ignore_array: ""
    baudrate: 115200        # скорость порта X3 Pro
    lidar_type: 1           # треугольный лидар
    device_type: 0          # подключение по последовательному порту
    sample_rate: 4          # 4000 измерений в секунду
    abnormal_check_count: 4
    fixed_resolution: true  # одинаковое число точек в каждом скане (нужно для
SLAM)
    reversion: false
    inverted: true
    auto_reconnect: true
    isSingleChannel: true   # X3 Pro только передаёт данные, команд не принимает
    intensity: false
    support_motor_dtr: false
    angle_max: 180.0
    angle_min: -180.0
    range_max: 8.0
    range_min: 0.1
    frequency: 10.0
    invalid_range_is_inf: false
EOF
```

Параметр	Значение для X3 Pro
`baudrate`	115200 — скорость порта
`lidar_type: 1`	треугольный (триангуляционный) лидар, не TOF
`isSingleChannel: true`	односторонняя связь: лидар только передаёт данные и не отвечает на команды
`sample_rate: 4`	4 тыс. измерений в секунду
`fixed_resolution: true`	одинаковое число точек в каждом скане — обязательно для построения карты (раздел 15)
`range_min` / `range_max`	0,1 … 8 м — рабочая дальность
`frame_id: laser_frame`	имя системы координат лидара

14.5. Запуск драйвера

```
ros2 run ydlidar_ros2_driver ydlidar_ros2_driver_node --ros-args --params-file
~/ydlidar_ros2_ws/x3pro.yaml
```

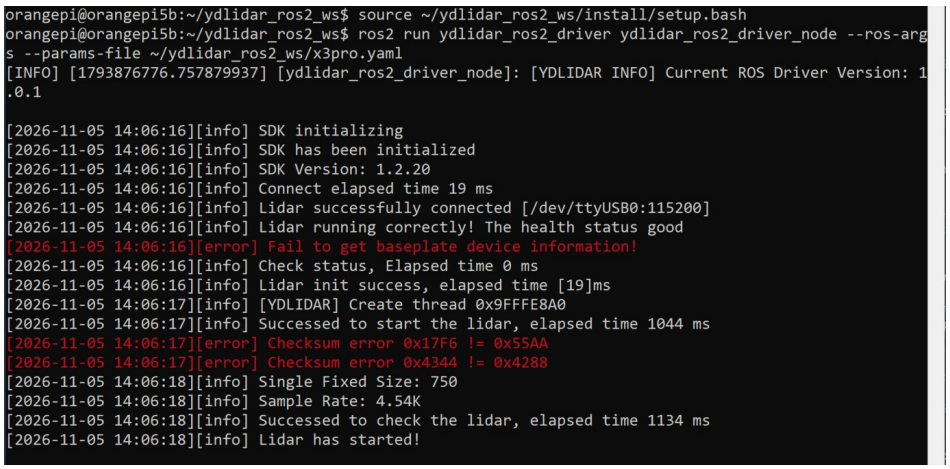


Рис. 36. Драйвер запущен: Lidar has started!, 750 точек за оборот, 4,54 тыс. измерений/с

Драйвер сообщает **«Lidar has started!»**, частоту измерений (4,54K) и число точек за оборот (750). Красные строки при старте не страшны:

- **Fail to get baseplate device information** — X3 Pro работает в одну сторону и не отвечает на запрос информации об устройстве;
- **Checksum error** (одна-две строки) — драйвер начал читать поток с середины пакета. Плохо, только если такие ошибки идут постоянно.

Во втором терминале (Bitvise → New terminal console, затем `su - orangepi`) проверьте частоту публикации скана:

```
ros2 topic hz /scan
```

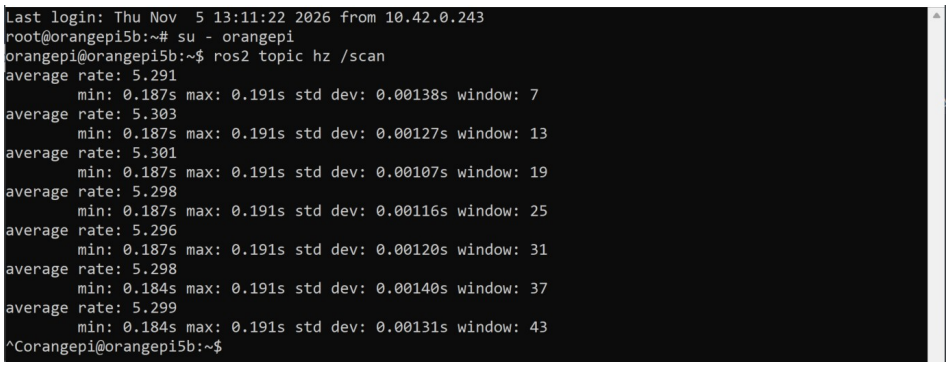


Рис. 37. Скан публикуется с частотой 5,3 Гц — это частота вращения лидара

Частота **5,3 Гц** очень стабильна (разброс около 1 мс) — это и есть частота вращения головки лидара.

14.6. RViz2

RViz2 запускается на рабочем столе VNC (раздел 11). Во втором терминале:

```
LIBGL_ALWAYS_SOFTWARE=1 DISPLAY=:1 rviz2
```

`DISPLAY=:1` — вывести окно на рабочий стол VNC; `LIBGL_ALWAYS_SOFTWARE=1` — программная отрисовка 3D (в сеансе VNC нет доступа к видеоускорителю; скорости хватает — около 30 кадров/с). Настройка окна:

1. Слева **Global Options** → **Fixed Frame**: заменить `map` на `laser_frame`.
2. Внизу **Add** → **By topic** → **/scan** → **LaserScan** → **OK**.
3. В пункте **LaserScan** → **Topic** → **Reliability Policy** выбрать **Best Effort** — драйвер публикует скан в режиме «данные датчика», и с настройкой по умолчанию (Reliable) точки не появятся.
4. **Size (m)** = `0.03` — точки крупнее и заметнее.

5. Справа **Views** → **Type** → **TopDownOrtho** — вид сверху без перспективы.

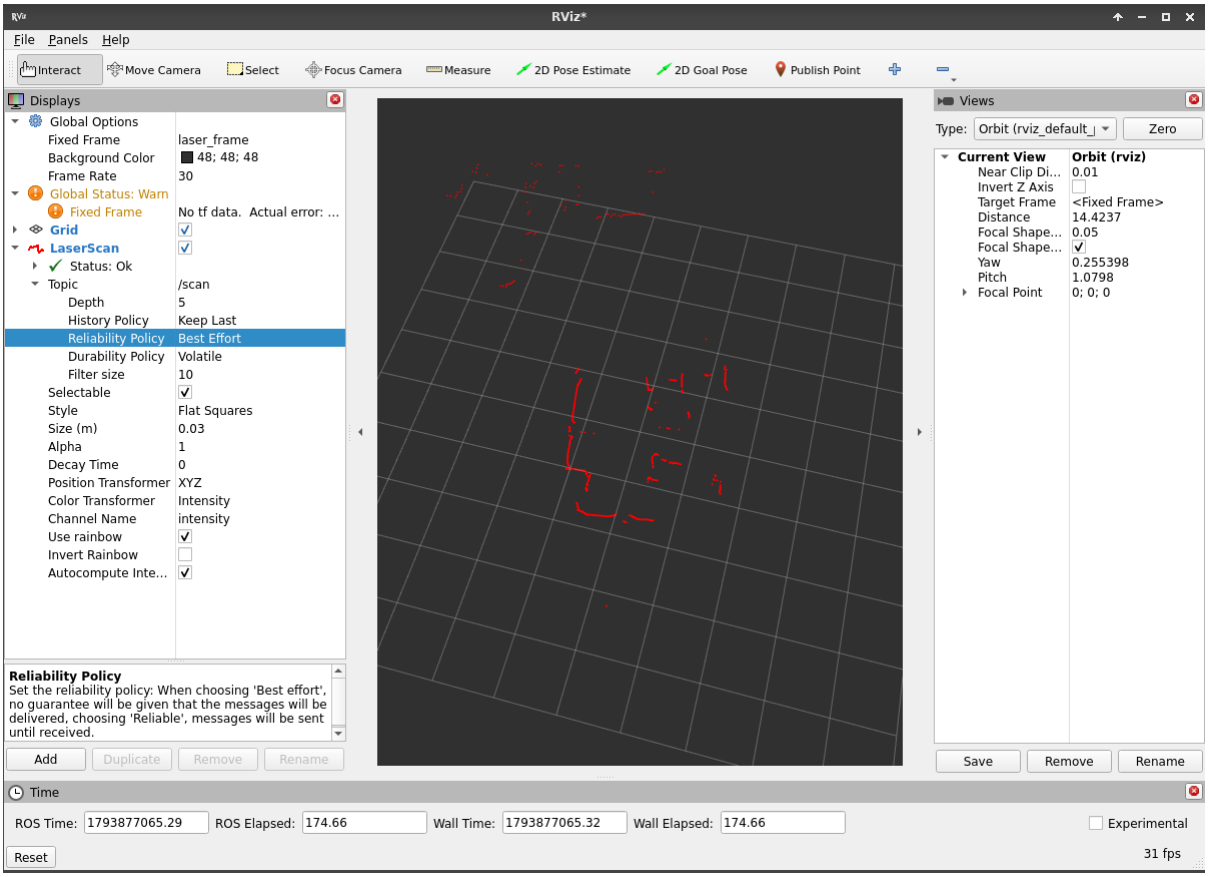


Рис. 38. Скан лидара в RViz2 (LaserScan: Status Ok)

Предупреждение **Fixed Frame: No tf data** при Fixed Frame = `laser_frame` не мешает отображению. Оно исчезнет, когда появится связь систем координат «робот → лидар» (раздел 15).

15. Построение карты помещения (SLAM)

SLAM (Simultaneous Localization And Mapping) — алгоритм, который по последовательным сканам одновременно строит карту и определяет положение лидара на ней. В ROS 2 стандартный инструмент — **slam_toolbox**.

slam_toolbox ожидает **одометрию** — данные о перемещении робота (обычно от энкодеров колёс). Пока у нас есть только лидар, поэтому одометрию вычисляем по самим сканам пакетом **rf2o_laser_odometry**: он сравнивает соседние сканы и определяет смещение лидара. Цепочка данных и систем координат:

Программа	Получает	Выдаёт
ydlidar_ros2_driver	данные лидара по USB	скан `/scan` в системе `laser_frame`
static_transform_publisher	—	положение лидара на роботе: `base_link → laser_frame`
rf2o_laser_odometry	`/scan`	одометрию `/odom` и `odom → base_link`
slam_toolbox	`/scan` и `odom → base_link`	карту `/map` и `map → odom`
RViz2	всё перечисленное	изображение карты, скана и

Программа	Получает	Выдаёт
		положения лидара

15.1. Установка

Под `root`:

```
apt install -y ros-humble-slam-toolbox ros-humble-nav2-map-server
```

`nav2-map-server` нужен для сохранения готовой карты в файл. Пакет rf2o собирается в нашей рабочей области (под `orangepi`):

```
su - orangepi
cd ~/ydlidar_ros2_ws/src
git clone -b ros2 https://github.com/MAPIRlab/rf2o_laser_odometry.git || git
clone https://github.com/MAPIRlab/rf2o_laser_odometry.git
cd ~/ydlidar_ros2_ws
colcon build --symlink-install
source ~/ydlidar_ros2_ws/install/setup.bash
```

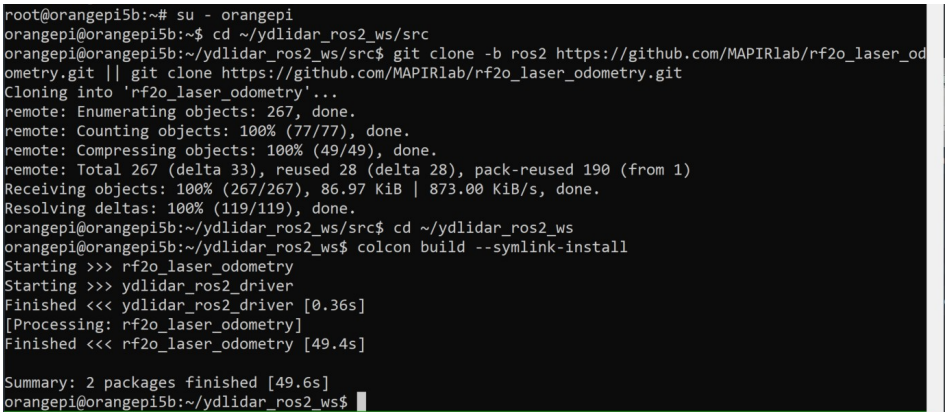


Рис. 39. rf2o собран: Summary: 2 packages finished

> **Важно.** После сборки нового пакета выполните `source ~/ydlidar_ros2_ws/install/setup.bash` (или откройте новый терминал) — иначе ROS 2 его не найдёт: `package 'rf2o_laser_odometry' not found`.

15.2. Настройки

Драйвер лидара: параметр `fixed_resolution: true` (уже задан в файле из раздела 14.4) — slam_toolbox требует одинаковое число точек в каждом скане.

Одометрия rf2o — файл `~/ydlidar_ros2_ws/rf2o.yaml`:

```
cat > ~/ydlidar_ros2_ws/rf2o.yaml << 'EOF'
rf2o_laser_odometry:
  ros__parameters:
    laser_scan_topic: /scan
    odom_topic: /odom
    publish_tf: true
    base_frame_id: base_link
    odom_frame_id: odom
    init_pose_from_topic: ""
    freq: 5.0
EOF
```

`init_pose_from_topic: ""` — обязательно пустая строка: иначе rf2o будет ждать начальное положение из другого топика и не запустится.

slam_toolbox — берём стандартные настройки и меняем систему координат робота, дальность лидара и порог обновления карты:

```
cp /opt/ros/humble/share/slam_toolbox/config/mapper_params_online_async.yaml
~/ydlidar_ros2_ws/slam.yaml
sed -i -e 's/base_footprint/base_link/' \
       -e 's/max_laser_range: .*/max_laser_range: 8.0/' \
       -e 's/minimum_travel_distance: .*/minimum_travel_distance: 0.1/' \
       -e 's/minimum_travel_heading: .*/minimum_travel_heading: 0.1/' \
       ~/ydlidar_ros2_ws/slam.yaml
grep -E "base_frame|max_laser_range|minimum_travel" ~/ydlidar_ros2_ws/slam.yaml
```

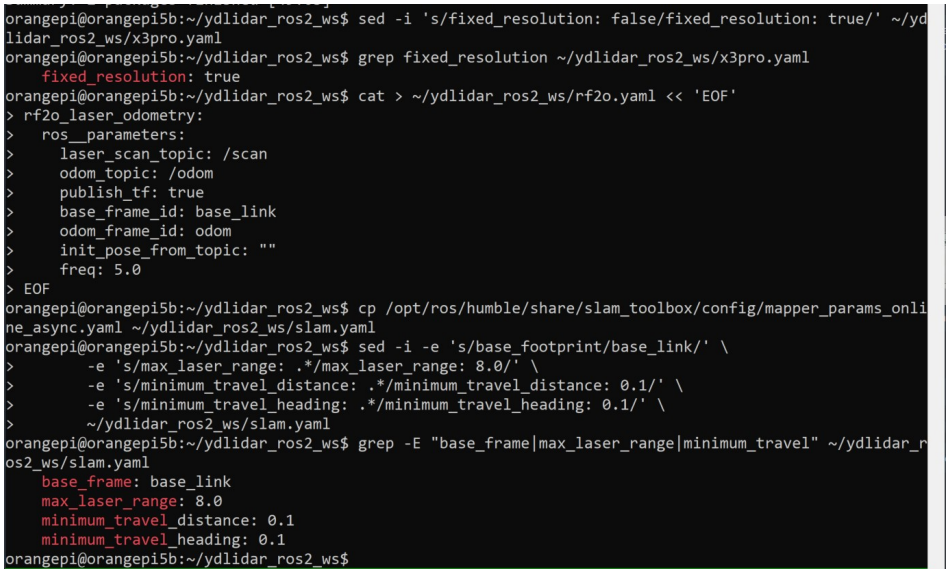


Рис. 40. Настройки rf2o и slam_toolbox

Карта будет обновляться после каждых 10 см перемещения или поворота на 0,1 рад (около 6°).

16. Запуск одной командой

Все программы (драйвер, связь систем координат, одометрия, SLAM, RViz2) запускаются одним файлом запуска (launch-файлом) `~/ydlidar_ros2_ws/slam_launch.py`. Готовый файл лежит в папке с инструкцией:

```
cat > ~/ydlidar_ros2_ws/slam_launch.py << 'EOF'
"""Лидар YDLIDAR X3 Pro + одометрия rf2o + slam_toolbox + подписи расстояний +
RViz2.

Запуск:   ros2 launch ~/ydlidar_ros2_ws/slam_launch.py
Без RViz: ros2 launch ~/ydlidar_ros2_ws/slam_launch.py rviz:=false
"""
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, ExecuteProcess,
IncludeLaunchDescription, TimerAction
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

WS = os.path.expanduser('~/ydlidar_ros2_ws')
RVIZ_CFG = os.path.join(WS, 'slam.rviz')

def generate_launch_description():
    slam = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(
            get_package_share_directory('slam_toolbox'), 'launch',
'online_async_launch.py')),
        launch_arguments={'slam_params_file': os.path.join(WS, 'slam.yaml'),
                          'use_sim_time': 'false'}.items())

    rviz = Node(
        package='rviz2', executable='rviz2', name='rviz2', output='log',
        arguments=['-d', RVIZ_CFG] if os.path.exists(RVIZ_CFG) else [],
        additional_env={'DISPLAY': ':1',               # рабочий стол VNC
                        'LIBGL_ALWAYS_SOFTWARE': '1'},  # программная отрисовка
3D
        condition=IfCondition(LaunchConfiguration('rviz')))

    return LaunchDescription([
        DeclareLaunchArgument('rviz', default_value='true',
                              description='Запускать RViz2 (true/false)'),
        # драйвер лидара -> топик /scan
        Node(package='ydlidar_ros2_driver',
executable='ydlidar_ros2_driver_node',
             name='ydlidar_ros2_driver_node', output='screen',
             parameters=[os.path.join(WS, 'x3pro.yaml')]),
        # положение лидара на роботе: base_link -> laser_frame
        Node(package='tf2_ros', executable='static_transform_publisher',
name='base_to_laser',
             arguments=['--frame-id', 'base_link', '--child-frame-id',
'laser_frame']),
        # одометрия по сканам: odom -> base_link
        Node(package='rf2o_laser_odometry',
executable='rf2o_laser_odometry_node',
             name='rf2o_laser_odometry', output='screen',
             parameters=[os.path.join(WS, 'rf2o.yaml')],
             arguments=['--ros-args', '--log-level', 'warn']),
        # SLAM: карта /map и map -> odom (через 3 с, когда лидар раскрутится)
```

```
        TimerAction(period=3.0, actions=[slam]),
        # подписи расстояний для RViz2 (топик /lidar_info)
        ExecuteProcess(cmd=['python3', os.path.join(WS, 'lidar_markers.py')],
output='screen'),
        # RViz2 (через 5 с)
        TimerAction(period=5.0, actions=[rviz]),
    ])
EOF
```

- slam_toolbox стартует через 3 с, RViz2 — через 5 с, когда лидар уже раскрутится и начнёт публиковать скан;
- RViz2 открывается на рабочем столе VNC (`DISPLAY=:1`) с настройками из файла `slam.rviz`;
- `--log-level warn` убирает служебные сообщения rf2o, которые он печатает на каждом скане;
- параметр `rviz:=false` позволяет запускать всё без RViz2 — например, на роботе;
- узел `lidar_markers.py` выводит в RViz2 подписи расстояний (раздел 16.4).

16.1. Настройки RViz2 для карты

Один раз настройте RViz2 и сохраните настройки — дальше они будут загружаться автоматически. Запустите `ros2 launch ~/ydlidar_ros2_ws/slam_launch.py` (RViz2 откроется пустым) и в окне RViz2:

1. **Fixed Frame** → `map`.
2. **Add** → **By topic** → **/map** → **Map**. Если карта не появилась: **Map** → **Topic** → **Durability Policy** = **Transient Local**.
3. **Add** → **By topic** → **/scan** → **LaserScan**, **Reliability Policy** = **Best Effort**, **Size** = `0.03`.
4. **Add** → **By display type** → **TF** — оси покажут положение лидара на карте.
5. **Views** → **Type** → **TopDownOrtho** — вид сверху.
6. **File** → **Save Config As** → `/home/orangepi/ydlidar_ros2_ws/slam.rviz`.

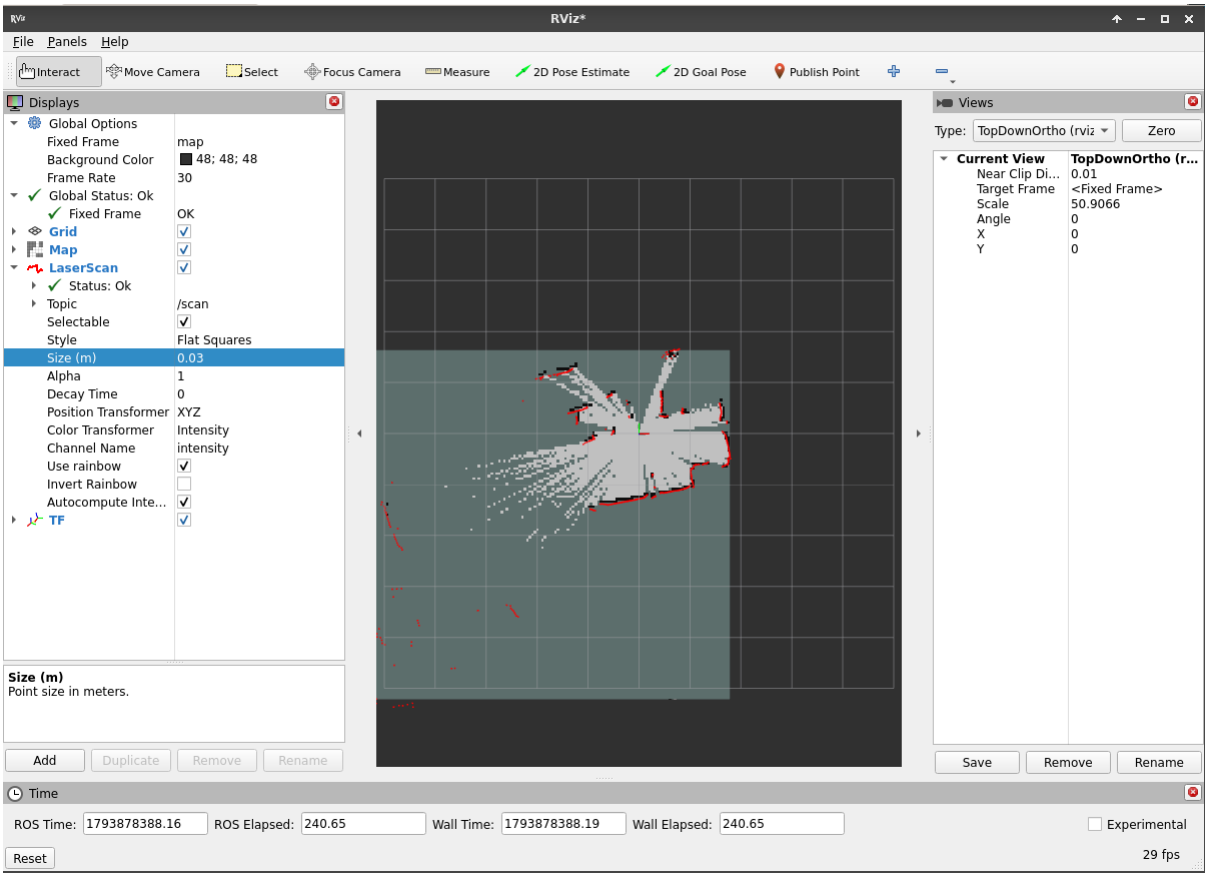


Рис. 41. Первая карта: светлым — свободная зона, чёрным — стены, красным — текущий скан

16.2. Короткие команды

```
echo "alias lidar_slam='ros2 launch ~/ydlidar_ros2_ws/slam_launch.py'" >>
~/.bashrc
echo "alias save_map='ros2 run nav2_map_server map_saver_cli -f ~/room_map'" >>
~/.bashrc
source ~/.bashrc
```

Команда	Что делает
`lidar_slam`	Лидар + одометрия + SLAM + подписи расстояний + RViz2 с картой
`lidar_slam rviz:=false`	То же без RViz2 (работа на роботе)
`save_map` (во втором терминале)	Сохранить карту: `~/room_map.pgm` (изображение) и `~/room_map.yaml` (масштаб и начало координат)
Ctrl+C в терминале запуска	Остановить всё, включая RViz2

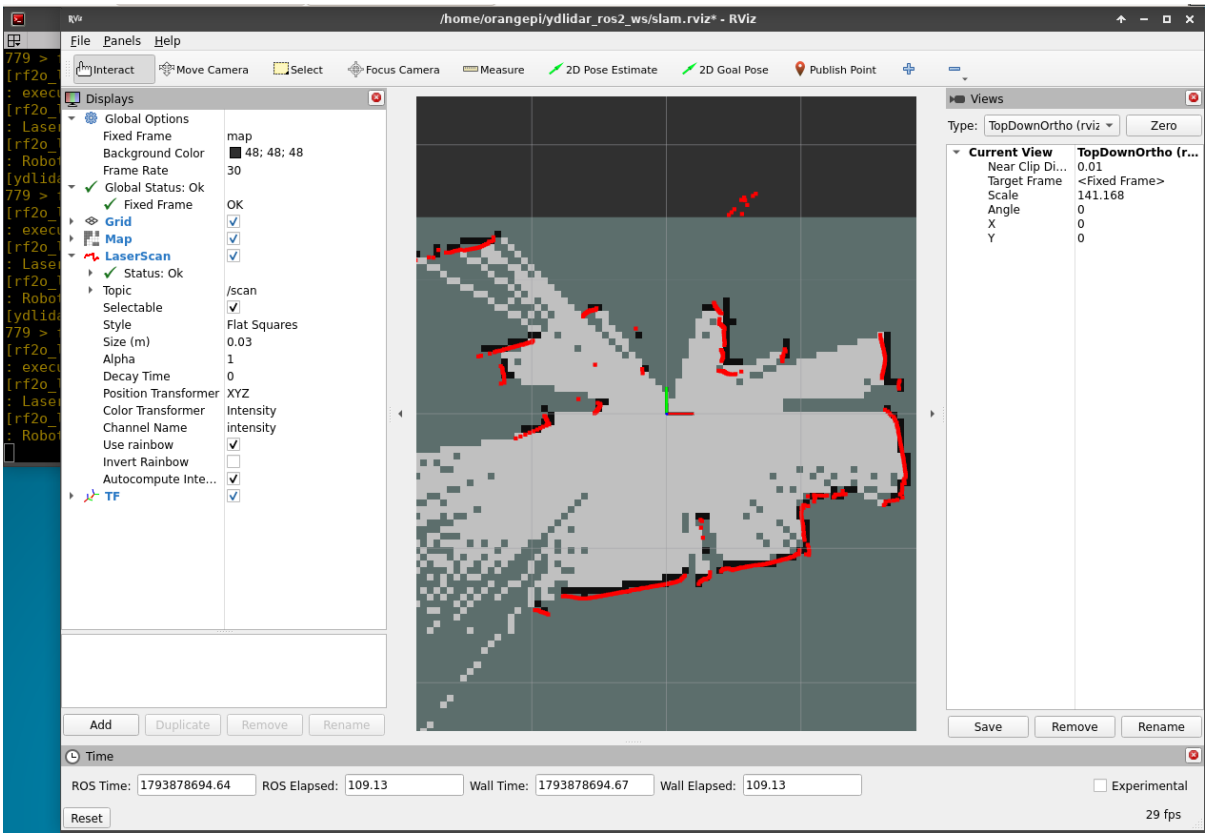


Рис. 42. Карта комнаты, построенная при запуске одной командой (lidar_slam)

16.3. Как построить карту всего помещения

- Стоя на месте, лидар рисует только видимую часть комнаты. Чтобы карта росла, **медленно** переносите плату с лидаром (удобно на подносе) или установите комплект на мобильную платформу.
- Двигайтесь шагом и поворачивайтесь плавно: при 5 оборотах в секунду резкие движения сбивают одометрию по сканам.
- Держите лидар **горизонтально** — при наклоне луч попадает в пол или потолок, и карта «плывёт».
- Пройдите по помещению и вернитесь в начальную точку — slam_toolbox заметит замкнутый маршрут и выровняет карту.
- Сохранённую карту (`room_map.pgm` + `room_map.yaml`) можно открыть в любом просмотрщике изображений и использовать позже для навигации робота.

16.4. Подписи расстояний в RViz2

RViz2 может показывать не только точки, но и подписи и линии — так называемые **маркеры** (Marker, MarkerArray). Небольшой узел ROS 2 `lidar_markers.py` получает скан `/scan` и публикует в топик `/lidar_info`:

- **красную линию** от лидара до ближайшего препятствия, красный шарик на нём и **жёлтую подпись** с расстоянием (например, `0.33m`);
- **белые подписи** с минимальным расстоянием в каждом из 8 секторов по 45° — у ближайшей точки сектора.

Подписи, которые оказались бы ближе 0,45 м друг к другу (параметр `MIN_GAP`), не ставятся — остаётся более близкая к лидару. В подписях используется латинская `m`: шрифт RViz2 может не отображать кириллицу.

Создайте файл `~/ydlidar_ros2_ws/lidar_markers.py` (готовый файл лежит в папке с инструкцией):

```
cat > ~/ydlidar_ros2_ws/lidar_markers.py << 'EOF'
#!/usr/bin/env python3
"""Расстояния лидара в RViz2: ближайшее препятствие и минимумы по 8 секторам."""
import math
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from rclpy.duration import Duration
from sensor_msgs.msg import LaserScan
from visualization_msgs.msg import Marker, MarkerArray
from geometry_msgs.msg import Point
from std_msgs.msg import ColorRGBA

SECTORS = 8          # секторы по 45°, сектор 0 — вперёд (ось X)
MIN_GAP = 0.45       # подписи ближе этого расстояния (м) друг к другу не ставим
RED = ColorRGBA(r=1.0, g=0.2, b=0.2, a=1.0)
YELLOW = ColorRGBA(r=1.0, g=1.0, b=0.2, a=1.0)
WHITE = ColorRGBA(r=1.0, g=1.0, b=1.0, a=1.0)


class LidarMarkers(Node):
    def __init__(self):
        super().__init__('lidar_markers')
        self.pub = self.create_publisher(MarkerArray, 'lidar_info', 10)
        self.create_subscription(LaserScan, 'scan', self.on_scan,
qos_profile_sensor_data)

    def marker(self, scan, mid, mtype, color):
        m = Marker()
        m.header.frame_id = scan.header.frame_id   # время 0 = последнее
известное положение
        m.ns, m.id, m.type, m.action = 'lidar_info', mid, mtype, Marker.ADD
        m.pose.orientation.w = 1.0
        m.color = color
        m.lifetime = Duration(seconds=0.3).to_msg()  # старая подпись исчезает,
если не обновилась
        return m

    def text(self, scan, mid, x, y, txt, color, size):
        m = self.marker(scan, mid, Marker.TEXT_VIEW_FACING, color)
        m.pose.position.x, m.pose.position.y, m.pose.position.z = x, y, 0.1
        m.scale.z = size                           # высота букв, м
        m.text = txt
        return m

    def on_scan(self, scan):
        # точки (расстояние, угол) с допустимым расстоянием
        pts = [(r, scan.angle_min + i * scan.angle_increment)
               for i, r in enumerate(scan.ranges)
               if math.isfinite(r) and scan.range_min <= r <= scan.range_max]
        if not pts:
            return
        arr = MarkerArray()
        placed = []                                # координаты уже поставленных
подписей
```

```
        # ближайшее препятствие: линия, точка и подпись
        r, a = min(pts)
        x, y = r * math.cos(a), r * math.sin(a)
        line = self.marker(scan, 0, Marker.LINE_STRIP, RED)
        line.scale.x = 0.02                         # толщина линии, м
        line.points = [Point(), Point(x=x, y=y)]
        dot = self.marker(scan, 1, Marker.SPHERE, RED)
        dot.pose.position.x, dot.pose.position.y = x, y
        dot.scale.x = dot.scale.y = dot.scale.z = 0.08
        arr.markers += [line, dot, self.text(scan, 2, x, y + 0.15, f'{r:.2f}m',
YELLOW, 0.14)]
        placed.append((x, y + 0.15))                # место жёлтой подписи

        # минимальное расстояние в каждом секторе
        best = {}
        for r, a in pts:
            k = int(((math.degrees(a) + 22.5) % 360) // 45)
            if k not in best or r < best[k][0]:
                best[k] = (r, a)
        for k in sorted(best, key=lambda k: best[k][0]):   # сначала самые
близкие
            r, a = best[k]
            x, y = r * math.cos(a), r * math.sin(a)
            if all(math.hypot(x - px, y - py) >= MIN_GAP for px, py in placed):
                arr.markers.append(self.text(scan, 10 + k, x, y, f'{r:.2f}',
WHITE, 0.12))
                placed.append((x, y))
        self.pub.publish(arr)


def main():
    rclpy.init()
    rclpy.spin(LidarMarkers())


if __name__ == '__main__':
    main()
EOF
```

- `qos_profile_sensor_data` — подписка на скан в режиме «данные датчика» (Best Effort), как публикует драйвер;
- `m.header.frame_id` без времени — маркер рисуется по последнему известному положению лидара. Если поставить на маркер время скана, RViz2 выдаст **Status: Error**: положение лидара на карте (`map → odom`) вычисляется с задержкой и для самого свежего скана ещё неизвестно;
- `lifetime = 0.3 c` — подпись исчезает, если перестала обновляться (например, сектор опустел).

Узел уже включён в файл запуска `slam_launch.py` (раздел 16) и стартует вместе с `lidar_slam`. Отдельно его можно запустить так: `python3 ~/ydlidar_ros2_ws/lidar_markers.py` (остановка — Ctrl+C; сообщение `KeyboardInterrupt` при этом — не ошибка).

В RViz2 один раз добавьте отображение: **Add → By topic → /lidar_info → MarkerArray → OK** — и сохраните настройки (**Ctrl+S**).

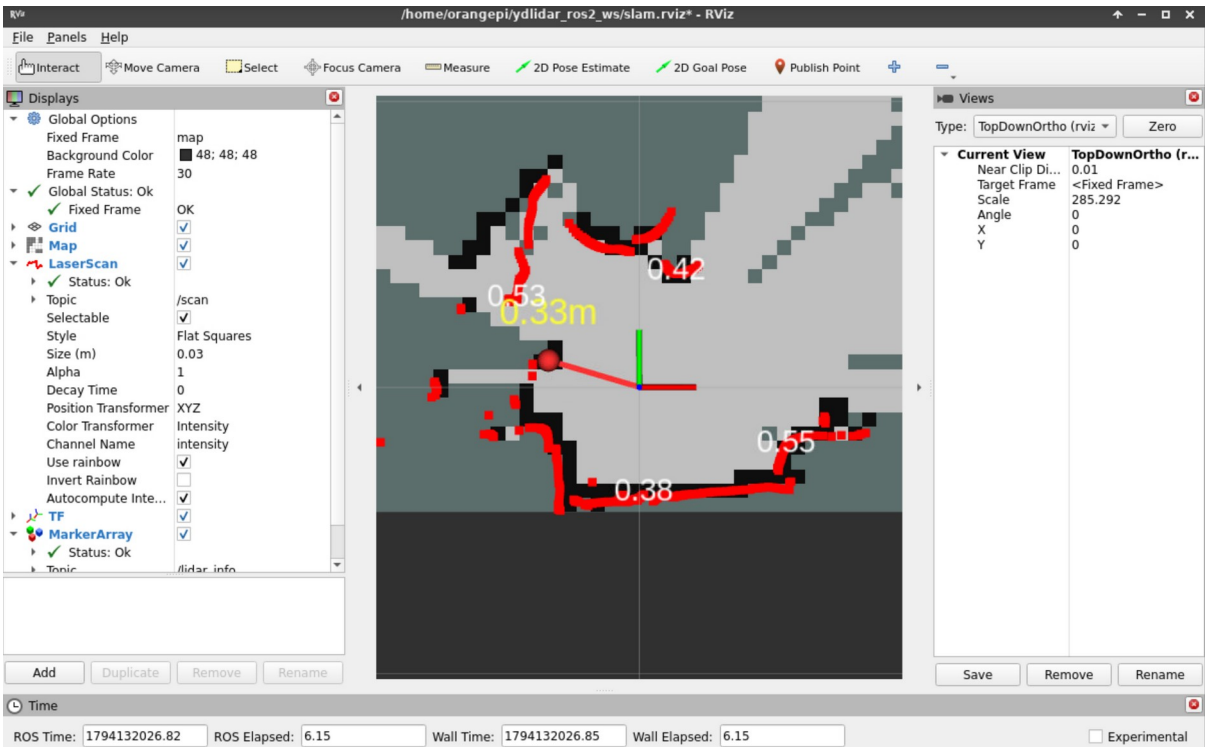


Рис. 43. Подписи расстояний: жёлтая — до ближайшего препятствия, белые — минимумы по секторам

Для разового измерения между двумя любыми точками в RViz2 есть кнопка **Measure** на панели инструментов.

17. Клонирование системы на другую плату

Когда одна плата полностью настроена (система, VNC, точка доступа, ROS 2, драйвер лидара, программы), её eMMC можно сохранить в файл-образ и записать на другие такие же платы — без повторения всех шагов инструкции.

Работающую систему копировать нельзя: файлы меняются во время чтения, и копия получится повреждённой. Поэтому плату загружают с карты microSD с **официальным образом Orange Pi** (той же, с которой ставили систему, раздел 3) — тогда eMMC не используется и копия получается целостной. Эта же карта затем служит для записи образа на новую плату.

Что нужно	Зачем
Карта microSD с официальным образом Orange Pi (лучше 64 ГБ)	Загрузка платы «со стороны», хранение и перенос образа (нужно около 5 ГБ свободного места)
Ethernet-кабель	На системе с карты нет точки доступа OrangePi5B — подключение через роутер
Новая плата Orange Pi 5B с eMMC не меньше, чем у исходной	Образ занимает весь объём исходной eMMC (29,1 ГБ)

17.1. Подготовка исходной платы

На работающей (настроенной) плате под `root` удалите лишнее и заполните свободное место нулями. Нули сжимаются почти до нуля, поэтому архив получится в разы меньше:

```
apt clean
journalctl --vacuum-time=1s
rm -rf /home/orangepi/.cache/pip
dd if=/dev/zero of=/zero.fill bs=4M status=progress; sync; rm -f /zero.fill
ls /zero.fill        # ожидается: No such file or directory
df -h /
```

Заполнение занимает около 10 минут и заканчивается ошибкой **No space left on device** — так и должно быть: свободное место заполнено, после чего временный файл сразу удаляется. `df` должен снова показать прежнее занятое место (у нас 6,1 ГБ из 29 ГБ).

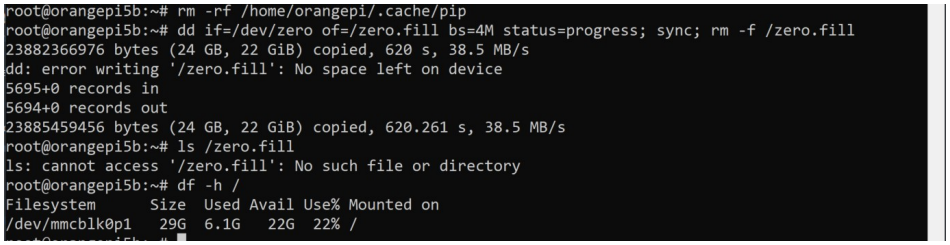


Рис. 44. Свободное место заполнено нулями, временный файл удалён

Выключите плату: `poweroff`.

17.2. Загрузка с карты microSD

1. Вставьте в выключенную плату карту microSD с официальным образом Orange Pi.
2. Подключите Ethernet-кабель и включите плату.
3. Найдите IP-адрес платы в роутере и подключитесь по SSH как `root` / `orangepi`. Bitvise предупредит о другом ключе сервера — у системы на карте свой ключ, примите его.
4. Убедитесь, что система загрузилась **с карты**:

```
lsblk
findmnt /dev/mmcblk0p1
df -h /
```

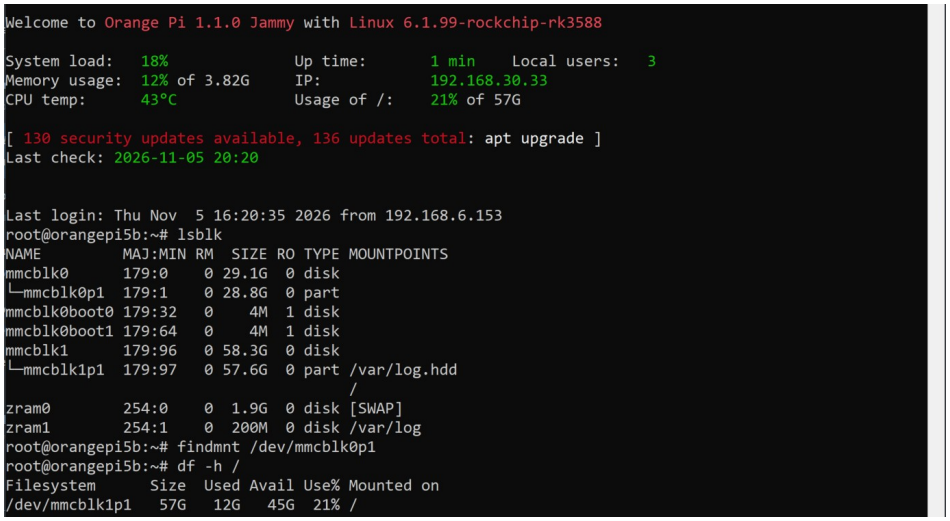


Рис. 45. Система загружена с карты: корень «/» на mmcblk1p1, eMMC (mmcblk0) не смонтирована

- корень `/` — на `mmcblk1p1` (карта, 57 ГБ);
- `findmnt /dev/mmcblk0p1` ничего не выводит — eMMC не используется;
- свободного места на карте не меньше 5 ГБ.

> **Внимание!** Если корень `/` на `mmcblk0p1` — плата загрузилась с eMMC, а не с карты. Снимать образ в этом случае нельзя: проверьте, что карта вставлена до конца, и перезагрузите плату.

17.3. Снятие образа

Программа `pigz` сжимает данные на всех 8 ядрах процессора. Работаем в tmux:

```
apt install -y pigz
tmux new -s img
dd if=/dev/mmcblk0 bs=4M status=progress | pigz -6 > /root/opi5b_lidar.img.gz
sync
ls -lh /root/opi5b_lidar.img.gz
```

Затем проверьте целостность архива и сохраните его контрольную сумму:

```
pigz -t /root/opi5b_lidar.img.gz && echo "АРХИВ ЦЕЛ"
cd /root && sha256sum opi5b_lidar.img.gz > opi5b_lidar.img.gz.sha256
cat opi5b_lidar.img.gz.sha256
```

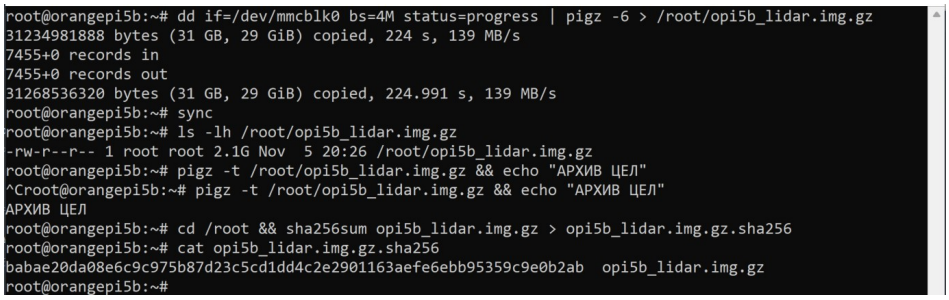


Рис. 46. Образ снят: 29,1 ГБ eMMC за 3,7 мин сжаты в файл 2,1 ГБ; архив цел, контрольная сумма сохранена

В нашем случае все 29,1 ГБ eMMC прочитаны за 3,7 минуты (139 МБ/с), а сжатый образ занял всего **2,1 ГБ** — благодаря заполнению свободного места нулями (раздел 17.1).

17.4. Копирование образа на компьютер

Сохраните образ на компьютере как эталон. В окне **SFTP** Bitvise откройте на плате папку `/root` и перетащите оба файла — `opi5b_lidar.img.gz` и `opi5b_lidar.img.gz.sha256` — в папку на компьютере. Копирование 2 ГБ по сети занимает несколько минут.

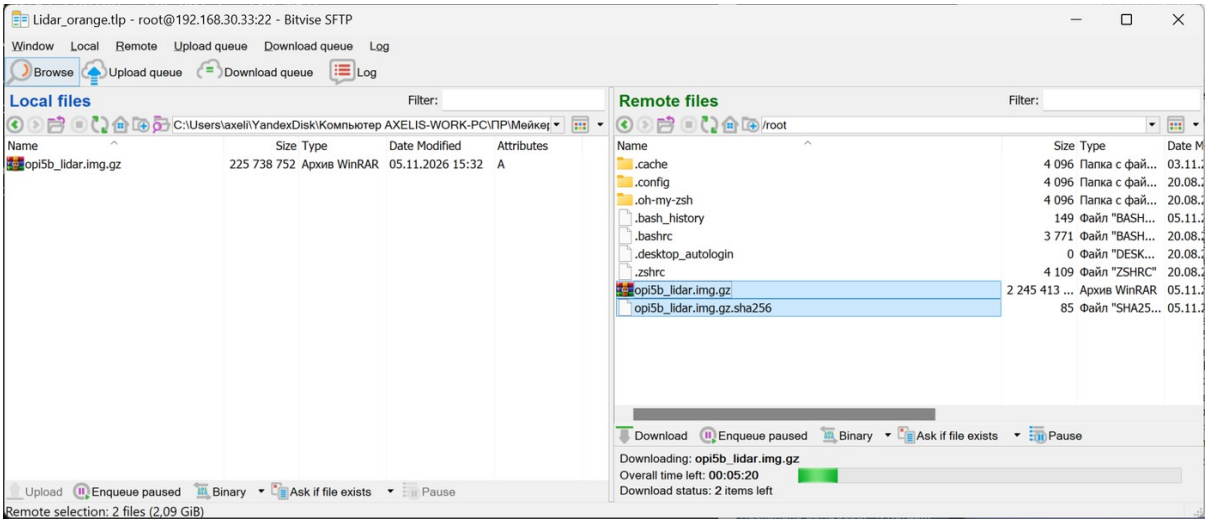


Рис. 47. Копирование образа на компьютер через SFTP в Bitvise

Проверьте, что копия совпадает с оригиналом. В PowerShell, в папке с файлом:

```
Get-FileHash .\opi5b_lidar.img.gz -Algorithm SHA256
```

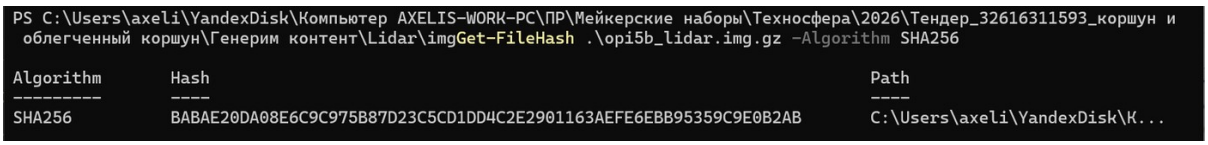


Рис. 48. Контрольная сумма копии на компьютере совпадает с суммой на плате

Значение должно совпасть с содержимым файла `.sha256` (Windows выводит его заглавными буквами — регистр не важен).

> **Копирование с MOC ОС.** Пользователь `orangepi` не имеет доступа к папке `/root`. Чтобы забрать образ через Dolphin, сначала переложите файлы: `mv /root/opi5b_lidar.img.gz* /home/orangepi/ && chown orangepi: /home/orangepi/opi5b_lidar.img.gz*`. Проверка суммы в MOC ОС: `sha256sum -c opi5b_lidar.img.gz.sha256`.

После копирования верните исходную плату в работу: `poweroff`, дождитесь, пока погаснет зелёный светодиод, **выньте карту** и включите плату — она загрузится с eMMC в прежнем виде.

17.5. Запись образа на новую плату

На новой плате на eMMC стоит заводской Android, но с картой официального образа она загружается так же, как первая (раздел 4).

1. Вставьте карту с образом (файл `/root/opi5b_lidar.img.gz` уже на ней) в новую плату, подключите Ethernet и включите.
2. Подключитесь по SSH как `root` / `orangepi` (IP — в роутере).
3. Проверьте устройства, наличие образа и его контрольную сумму:

```
lsblk
ls -lh /root/opi5b_lidar.img.gz
sha256sum -c /root/opi5b_lidar.img.gz.sha256
```

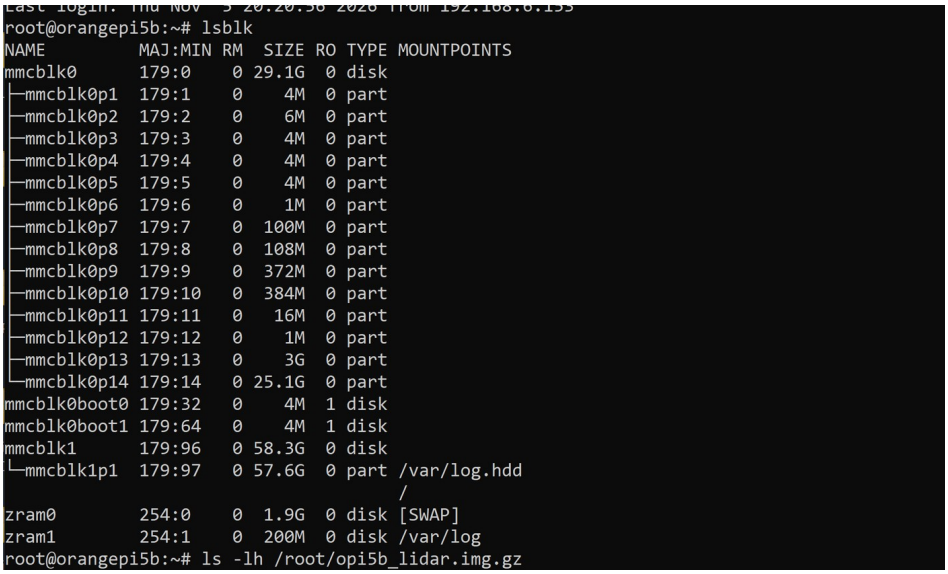


Рис. 49. Новая плата: eMMC 29,1 ГБ с 14 разделами Android (mmcblk0), система работает с карты (mmcblk1), образ цел (OK)

Корень `/` должен быть на `mmcblk1` (карта), а `mmcblk0` — eMMC такого же или большего размера. Контрольная сумма — `OK`. Затем запишите образ на eMMC:

```
tmux new -s img
gunzip -c /root/opi5b_lidar.img.gz | dd of=/dev/mmcblk0 bs=4M status=progress
conv=fsync
sync
```

Запись 29,1 ГБ на eMMC занимает около 14 минут (скорость записи около 38 МБ/с). Для надёжности сравните записанное с образом байт в байт (около 5 минут):

```
gunzip -c /root/opi5b_lidar.img.gz | cmp - /dev/mmcblk0 && echo "ЗАПИСЬ
СОВПАДАЕТ"
poweroff
```

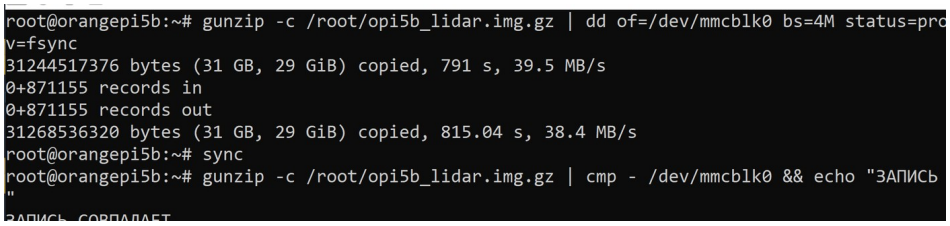


Рис. 50. Образ записан на eMMC новой платы (31 268 536 320 байт) и проверен: ЗАПИСЬ СОВПАДАЕТ

> **Осторожно!** Запись идёт на `/dev/mmcblk0` (eMMC). Если перепутать с картой (`mmcblk1`), будет стёрта система, с которой работает плата, вместе с образом.

Выньте карту и включите плату. Загрузится копия исходной системы со всеми настройками: удалённый рабочий стол, точка доступа, ROS 2, драйвер лидара, программы и команда `lidar_slam`. Подключиться можно сразу по Wi-Fi `OrangePi5B` к адресу `10.42.0.1` (если исходная плата выключена) или по Ethernet.

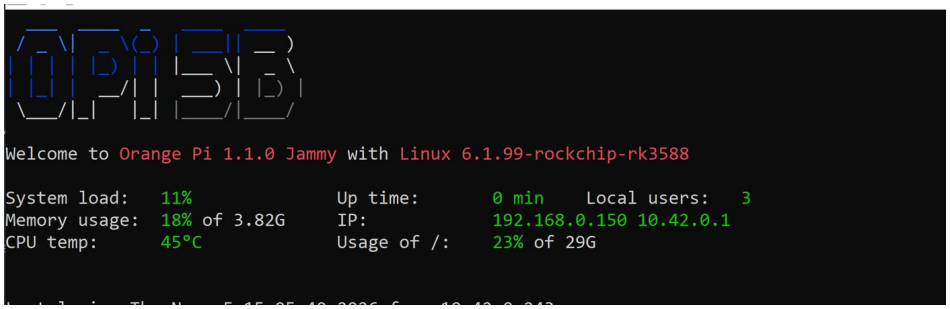


Рис. 51. Новая плата загрузилась с eMMC: точка доступа 10.42.0.1 поднялась, история входов перенесена из образа

Если eMMC новой платы больше (Orange Pi 5B выпускается с eMMC на 32, 64, 128 и 256 ГБ), раздел займёт только 29 ГБ. Растянуть его на весь объём:

```
growpart /dev/mmcblk0 1
resize2fs /dev/mmcblk0p1
df -h /
```

`growpart` входит в пакет `cloud-guest-utils` (`apt install -y cloud-guest-utils`). На плату с **меньшей** eMMC образ не поместится.

17.6. Как сделать копию уникальной

У копии будут те же имя компьютера, имя Wi-Fi сети, идентификатор системы и ключи SSH, что у оригинала. Если две платы окажутся рядом или в одной сети, начнутся конфликты. На новой плате под `root` (подключение — через Wi-Fi `OrangePi5B` по адресу `10.42.0.1` при выключенной исходной плате, или через Ethernet по IP из роутера):

```
# имя компьютера
hostnamectl set-hostname orangepi5b-2
sed -i 's/orangepi5b/orangepi5b-2/g' /etc/hosts

# имя Wi-Fi сети, чтобы не совпадало с другими платами
nmcli con modify OPi-Hotspot 802-11-wireless.ssid OrangePi5B-2

# уникальный идентификатор системы (иначе роутер может выдать платам один IP)
rm -f /etc/machine-id /var/lib/dbus/machine-id
systemd-machine-id-setup
ln -sf /etc/machine-id /var/lib/dbus/machine-id

# новые ключи SSH-сервера
rm -f /etc/ssh/ssh_host_*
dpkg-reconfigure openssh-server

# отдельный «канал» ROS 2, чтобы данные лидаров разных плат не смешивались
echo "ROS_DOMAIN_ID=2" >> /etc/environment

# проверка перед перезагрузкой
grep orangepi5b /etc/hosts; nmcli -g 802-11-wireless.ssid con show OPi-Hotspot;
cat /etc/machine-id
systemctl is-enabled ssh; systemctl is-active ssh

reboot
```

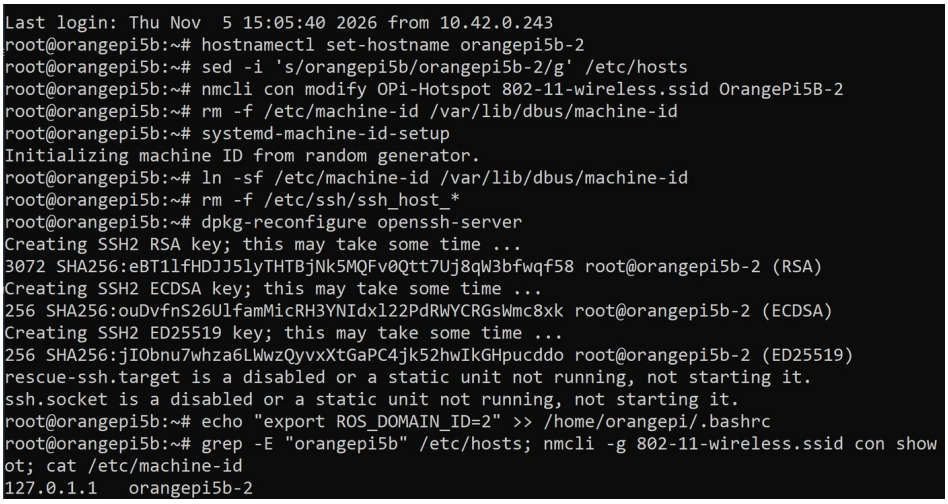


Рис. 52. Новое имя, SSID, machine-id и ключи SSH-сервера

Строки `rescue-ssh.target … not starting it` и `ssh.socket … not starting it` безобидны — это вспомогательные службы, которые здесь не используются; SSH-сервер работает как служба `ssh`. После перезагрузки подключите компьютер к Wi-Fi **OrangePi5B-2** (пароль прежний), подключитесь к `10.42.0.1` — Bitvise предупредит о смене ключа сервера, примите его и сохраните для новой платы отдельный профиль. Проверка:

```
hostname
systemctl is-active vncserver@1
grep ROS_DOMAIN_ID /etc/environment
```

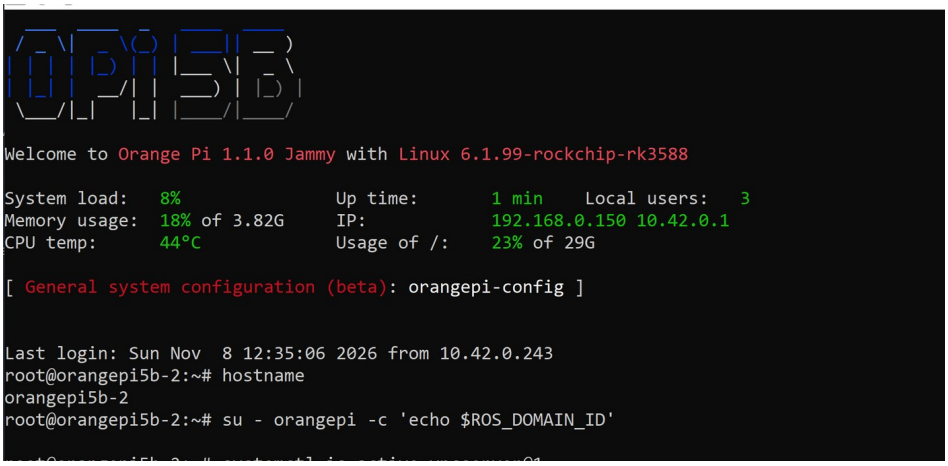


Рис. 53. После перезагрузки: имя orangepi5b-2, удалённый рабочий стол работает

Почему ROS_DOMAIN_ID — в /etc/environment, а не в ~/.bashrc. Строки, добавленные в конец `~/.bashrc`, действуют только в интерактивных терминалах: в Ubuntu файл начинается с проверки, которая для неинтерактивных команд (например, `su - orangepi -c …` или запуска через службу) прекращает его выполнение. Файл `/etc/environment` читается при любом входе в систему, поэтому номер домена действует везде.

Что меняем	Почему
Имя компьютера (`hostnamectl`)	Чтобы различать платы в роутере, в приглашении терминала и в программах
Имя Wi-Fi сети (SSID)	Две точки доступа с одинаковым именем мешают выбрать нужную плату
machine-id	По нему роутер выдаёт адрес — одинаковый machine-id может привести к одинаковому IP у двух плат
Ключи SSH-сервера	У каждой платы должен быть свой ключ — иначе нельзя отличить одну плату от другой
`ROS_DOMAIN_ID`	Платы в одной сети по умолчанию «видят» топики друг друга (`/scan`, `/map`) — номер домена их разделяет

Для каждой следующей платы увеличивайте номер: `orangepi5b-3`, `OrangePi5B-3`, `ROS_DOMAIN_ID=3` и т. д. Адрес платы в её Wi-Fi сети остаётся `10.42.0.1` — это не мешает, так как у каждой платы своя сеть. При первом подключении к новой плате Bitvise покажет предупреждение о ключе сервера — примите его; на MOC OC может понадобиться удалить старый ключ командой `ssh-keygen -R 10.42.0.1`.

18. Типичные проблемы

Проблема	Причина и решение
При установленной карте горит только красный светодиод	Загрузчик Android на eMMC мешает загрузке стороннего образа (например, Armbian). Использовать официальный образ Orange Pi (раздел 2) или очистить eMMC через режим MaskROM (RKDevTool).
Без карты загружается Android с сервисами Google	Это заводская система на eMMC. Перенести Linux на eMMC (раздел 6).
Не удаётся подключиться по SSH	Проверить IP-адрес (роутер, Advanced IP Scanner), огоньки на Ethernet-разъёме, порт 22, раскладку клавиатуры и

Проблема	Причина и решение
	Caps Lock при вводе пароля.
apt update: NO_PUBKEY … is not signed	Лишний репозиторий Docker на зеркале aliyun — удалить его (раздел 7.2).
SSH отключается во время обновления	Обновление libc6 перезапускает SSH. Проверить `uptime`, переподключиться, запускать длительные команды в tmux (раздел 7.3–7.6).
Waiting for cache lock	Работает другой процесс apt (фоновые обновления или прерванная по SSH команда). Дождаться, не прерывать.
SSH и VNC периодически рвутся, плата при этом не перезагружается (`uptime` большой)	Обрывы в сети между подсетями. Подключаться напрямую через точку доступа платы (раздел 9).
Плата внезапно перезагружается, журналы за последние минуты пропали	Просадка питания. Проверить блок питания (5 В / 4 А), кабель и разъём; тест stress-ng (раздел 8).
VNC: чёрный экран вместо рабочего стола	Второй сеанс Xfce не запускается рядом с сеансом на мониторе. В `~/.vnc/xstartup` должна быть строка `exec dbus-launch --exit-with-session startxfce4` (раздел 10.3).
KRDC: rfbInitClient failed	Клиент не поддерживает шифрование VeNCrypt. Добавить `-SecurityTypes VncAuth` в службу VNC (раздел 10.4).
RDP (mstsc, xfreerdp): ошибка 0x4 / Timeout waiting for activation	Несовместимость xrdp 0.9.17 с современными клиентами. Использовать VNC (раздел 10).
Нет /dev/ttyUSB0 или порт появляется и пропадает	Служба brltty перехватывает USB-UART адаптер — `apt remove -y brltty`, переподключить лидар. Проверить кабель и `dmesg | tail`.
Permission denied при открытии /dev/ttyUSB0	Пользователь не в группе dialout: `usermod -aG dialout orangepi`, затем выйти и войти заново.
Получено 0 байт, пакетов AA 55 нет	Проверить, вращается ли головка и надёжно ли вставлен шлейф в адаптер. Если мотор не крутится — не хватает питания по USB.
lidar_plot.py: ошибка «couldn't connect to display»	Не указан или не запущен дисплей VNC: запускать как `DISPLAY=:1 python3 ~/lidar_plot.py`, проверить `systemctl status vncserver@1`.
ros2 launch: package '…' not found	Пакет собран, но окружение не обновлено: `source ~/ydlidar_ros2_ws/install/setup.bash` или открыть новый терминал.
Драйвер: Fail to get baseplate device information, разовые Checksum error	Нормально для X3 Pro (односторонняя связь, старт с середины пакета).
RViz2: LaserScan без точек	LaserScan → Topic → Reliability Policy = Best Effort; проверить `ros2 topic hz /scan`.
RViz2: нет карты	Map → Topic → Durability Policy = Transient Local; Fixed Frame = map; убедиться, что slam_toolbox запущен.
RViz2: Fixed Frame — No tf	Нет связи систем координат. Запустить

Проблема	Причина и решение
data	static_transform_publisher (base_link → laser_frame) или весь slam_launch.py.
slam_toolbox: LaserRangeScan contains … range readings, expected …	Разное число точек в сканах — в x3pro.yaml должно быть `fixed_resolution: true`.
Карта «плывёт», стены двоятся	Слишком быстрое перемещение или наклон лидара. Двигаться медленнее, держать лидар горизонтально.
Образ: после записи плата не загружается с eMMC	Проверить, что запись шла на `mmcblk0` и завершилась без ошибок; проверить архив `pigz -t` и контрольную сумму; повторить запись.
Две платы получают один IP или мешают друг другу	Не выполнен раздел 17.6: сменить machine-id, имя, SSID, ROS_DOMAIN_ID.
ssh в MOC OC: WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED	Сменился ключ сервера (переустановка или другая плата с тем же адресом): `ssh-keygen -R 10.42.0.1`.
RViz2: MarkerArray — Status: Error	У маркеров стоит время скана, а положение лидара на карте ещё не вычислено. В lidar_markers.py указывать только `m.header.frame_id` (раздел 16.4).
lidar_slam: command not found	Команда задана для пользователя orangepi: выполнить `su - orangepi`.
Предупреждение Bitvise о смене ключа сервера	Система переустановлена — принять новый ключ.